

INHERITED FROM Helix Constitution

This module is a submodule of a Helix-family project (e.g. HelixCode, HelixAgent, ATMOSphere) that includes the Helix Constitution submodule at the parent's constitution/ path. All rules in constitution/CLAUDE.md and the constitution/Constitution.md it references (universal anti-bluff covenant §11.4, no-guessing mandate §11.4.6, credentials-handling mandate §11.4.10, host-session safety §12, data safety §9, mutation- paired gates §1.1) apply unconditionally to every change landed here. The module-specific rules below extend them — they never weaken any universal clause.

When this file disagrees with the constitution submodule, the constitution wins. Locate the constitution submodule from any arbitrary nested depth using its find_constitution.sh helper.

Canonical reference: <https://github.com/HelixDevelopment/HelixConstitution>

HelixCode Constitution

Inherits from: HelixConstitution/Constitution.md — all universal clauses in that document apply unconditionally. Rules below extend or tighten them; they MUST NOT weaken any inherited clause.

HelixCode Project Constitution

Version: 1.0.0 **Effective Date:** 2026-04-30 **Scope:** This Constitution applies to HelixCode and ALL its submodules
Authority: Cascaded from HelixAgent root governance with HelixCode-specific addenda

Preamble

HelixCode is an enterprise-grade distributed AI development platform. This Constitution establishes the non-negotiable rules that govern all development, testing, deployment, and

maintenance activities within the project. Every contributor, agent, and automated process MUST adhere to these rules. No exceptions.

CONST-001: No CI/CD Pipelines (Permanent)

No .github/workflows/, .gitlab-ci.yml, Jenkinsfile, .travis.yml, .circleci/, or any automated pipeline. No Git hooks. All builds and tests run manually or via Makefile/script targets.

Rationale: Manual execution ensures human oversight and prevents automated propagation of bluffs.

CONST-002: No Mocks in Production (Permanent)

CONST-002a: Production Code

Mocks, stubs, fakes, placeholder classes, TODO implementations are STRICTLY FORBIDDEN in production code. All production code is fully functional with real integrations.

CONST-002b: Test Code

Mocks/stubs/fakes MAY be used ONLY in unit tests (files ending _test.go run under `go test -short`).

Rationale: Production bluffs have repeatedly been discovered where features appeared implemented but were non-functional.

<<<<<< HEAD ## CONST-003: No HTTPS for Git (Permanent)

SSH URLs only (git@github.com:..., git@gitlab.com:..., etc.) for clones, fetches, pushes, and submodule updates. SSH keys are configured on every service.

CONST-004: No Manual Container Commands (Permanent)

Container orchestration is owned by the project's binary/orchestrator (e.g., make build → ./bin/<app>). Direct docker/podman start|stop|rm and docker-compose up|down are prohibited as workflows.

CONST-005: 100% Real Data for Non-Unit Tests

Beyond unit tests, all components MUST use actual API calls, real databases, live services. No simulated success. Fallback chains tested with actual failures.

Verification: Every integration/E2E test MUST connect to real services or skip (not fail) if unavailable.

CONST-006: Challenge Coverage (Permanent)

Every component MUST have Challenge scripts (./challenges/scripts/) validating real-life use cases. No false success — validate actual behavior, not return codes.

CONST-007: Health & Observability

Every service MUST expose health endpoints. Circuit breakers for all external dependencies. Prometheus / OpenTelemetry integration where applicable.

CONST-008: Documentation & Quality

Update CLAUDE.md, AGENTS.md, and relevant docs alongside code changes. Pass language-appropriate format/lint/security gates. Conventional Commits: <type>(<scope>): <description>.

CONST-009: Validation Before Release

Pass the project's full validation suite (`make ci-validate-all-equivalent`) plus all challenges (`./challenges/scripts/run_all_challenges.sh`).

CONST-010: Comprehensive Verification

Every fix **MUST** be verified from all angles: runtime testing (actual HTTP requests / real CLI invocations), compile verification, code structure checks, dependency existence checks, backward compatibility, and no false positives. Grep-only validation is **NEVER** sufficient.

CONST-011: Resource Limits for Tests & Challenges

ALL test and challenge execution **MUST** be strictly limited to 30-40% of host system resources. Use `GOMAXPROCS=2`, `nice -n 19`, `ionice -c 3, -p 1` for go test. Container limits required.

CONST-012: Bugfix Documentation

All bug fixes **MUST** be documented in `docs/issues/fixed/BUGFIXES.md` with root cause analysis, affected files, fix description, and a link to the verification test/challenge.

CONST-013: Real Infrastructure for All Non-Unit Tests

Mocks/fakes/stubs/placeholders **MAY** be used **ONLY** in unit tests. ALL other test types — integration, E2E, functional, security, stress, chaos, challenge, benchmark, runtime verification — **MUST** execute against **REAL** running systems with **REAL** containers, **REAL** databases, **REAL** services, and **REAL** HTTP calls.

CONST-014: Reproduction-Before-Fix (Mandatory)

Every reported error, defect, or unexpected behavior MUST be reproduced by a Challenge script BEFORE any fix is attempted. Sequence: 1. Write the Challenge first 2. Run it; confirm fail (it reproduces the bug) 3. Then write the fix 4. Re-run; confirm pass 5. Commit Challenge + fix together

The Challenge becomes the regression guard for that bug forever.

CONST-015: Concurrent-Safe Containers

Any struct field that is a mutable collection (map, slice) accessed concurrently MUST use thread-safe primitives. Bare `sync.Mutex` + map/slice combinations are prohibited for new code.

CONST-016: Definition of Done (Universal)

A change is NOT done because code compiles and tests pass. "Done" requires pasted terminal output from a real run.

- **No self-certification:** Words like *verified*, *tested*, *working*, *complete*, *fixed*, *passing* are forbidden in commits/PRs/replies unless accompanied by pasted output from a command that ran in that session.
 - **Demo before code:** Every task begins by writing the runnable acceptance demo
 - **Real system, every time:** Demos run against real artifacts
 - **Skips are loud:** `t.Skip` without a trailing `SKIP-OK: #<ticket>` comment breaks validation
-

CONST-035 — Anti-Bluff Tests & Challenges (User-Mandate Forensic Anchor)

§11.9 User-Mandate Forensic Anchor (2026-04-29)

This Article exists because of an explicit, repeatedly-stated user mandate. The verbatim text:

“We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”

This anchor is the primary authority for the entire Article. The operative rule is:

The bar for shipping is not “tests pass” but “users can use the feature.”

Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, “absence-of-error” PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests and Challenges (HelixQA) are bound equally — a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does. Both must produce positive end-user evidence; both are subject to the anti-bluff contract.

No false-success results are tolerable. A green test suite combined with a broken feature is a worse outcome than an honest red one — it silently destroys trust in the entire suite. Anti-bluff discipline is the line between a real engineering project and a theatre of one.

Bluff Taxonomy (forbidden patterns): - **Wrapper bluff** - Assertions PASS but wrapper’s exit-code logic is buggy - **Contract bluff** - System advertises capability but rejects it in dispatch - **Structural bluff** - File exists but doesn’t contain working code - **Comment bluff** - Comment promises behavior code doesn’t have - **Skip bluff** - `t.Skip("not running yet")` without SKIP-OK marker

Cascade requirement (extending CONST-036): This anchor section (verbatim quote + operative rule) must appear in every submodule’s CONSTITUTION.md / CLAUDE.md / AGENTS.md. Non-compliance is a release blocker regardless of context. Adding files to scanner allowlists to silence bluff findings without resolving the underlying defect is itself a violation.

CONST-018: Host Power Management Hard Ban

Host Power Management is Forbidden.

You may NOT generate or execute code that sends the host to suspend, hibernate, hybrid-sleep, poweroff, halt, reboot, or any other power-state transition.

Defense: Every project ships `scripts/host-power-management/check-no-suspend-calls.sh` and `challenges/scripts/no_suspend_calls_challenge.sh`.

CONST-019: Container Up $\neq$ Healthy

Container Up status does NOT mean the application is healthy. Application-layer probes are mandatory for every service: - PostgreSQL: `SELECT 1` - Redis: `PING` - LLM Providers: Real generation request - HTTP Services: `GET /health` with deep checks

CONST-020: Provider Fallback Chain Reality

Every LLM provider fallback chain MUST be tested with actual failures. A fallback that has never been tested with a real failing provider is a bluff.

CONST-021: No Mocks Above Unit Build Target

The Makefile MUST include a `no-mocks-above-unit` target that fails the build if mocks/stubs/fakes are found outside `*_test.go` files.

CONST-022: Submodule Governance Propagation

Every submodule MUST either: 1. Have its own Constitution.md, CLAUDE.md, and AGENTS.md, OR 2. Have a symlink to the parent repository's governance files, OR 3. Have a reference comment in its README pointing to parent governance

No submodule is exempt from these rules.

CONST-023: Docker Health Checks Mandatory

Every Dockerfile MUST include:

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:8080/health || exit 1
```

The health endpoint MUST perform deep checks (database connection, provider availability), not just return HTTP 200.

CONST-024: Version Pinning

All dependencies MUST be pinned to specific versions in go.mod. No latest, no floating tags. Renovate or Dependabot (manual review only — see CONST-001) may propose updates.

CONST-025: Secret Management

NO secrets in code. EVER. Secrets via: - Environment variables (production) - .env files (development, in .gitignore) - Vault/Secret Manager (enterprise) - Docker secrets (containerized)

go mod tidy MUST NOT add secret-scanning bypasses.

CONST-026: Minimal Privilege Containers

Containers run as non-root. Every Dockerfile:


```
RUN adduser -D -u 1001 helixcode
USER helixcode
```

CONST-027: Network Isolation

Container orchestration **MUST** use internal networks. Services communicate via named hosts, not exposed ports where possible.

CONST-028: Backup Before Destructive Operations

Every file editing tool **MUST** create backups before modification. The backup **MUST** be restorable.

CONST-029: Input Validation at All Boundaries

Every public function **MUST** validate inputs. No trust of caller-provided data. SQL injection, path traversal, command injection **MUST** be impossible by design.

CONST-030: Graceful Degradation

When external services are unavailable, the system **MUST** degrade gracefully: - Return partial results where possible - Queue operations for retry - Inform user of degraded state - NEVER crash or hang indefinitely

CONST-031: Audit Trail

Every significant operation **MUST** be logged with: - Timestamp - User identity - Operation type - Success/failure status - Resource affected

Log retention: 90 days minimum.

CONST-032: Emergency Stop

Every long-running or distributed operation MUST support cancellation via `context.Context`. Users MUST be able to interrupt any operation.

CONST-033: Data Integrity

Database writes MUST be transactional. Partial writes MUST be rolled back. Consistency checks MUST run periodically.

CONST-034: API Stability

Public APIs maintain backward compatibility within major versions. Deprecation requires: - 6-month notice - Migration guide - Compatibility shim

CONST-035: End-User Usability Mandate (2026-04-29 Strengthening)

A test or Challenge that PASSES is a CLAIM that the tested behavior **works for the end user of the product**.

The HelixAgent project has repeatedly hit the failure mode where every test ran green AND every Challenge reported PASS, yet most product features did not actually work. This MUST NOT recur in HelixCode.

Every PASS result MUST guarantee: a. **Quality** - correct behavior under real inputs, edge cases, concurrency b. **Completion** - wired end-to-end with no stub/placeholder gaps c. **Full usability** - a user following documented request shapes SUCCEEDS

A passing test that doesn't certify all three is a **bluff** and MUST be tightened.

Bluff taxonomy (each pattern observed and now forbidden): - **Wrapper bluff** - assertions PASS but wrapper's exit-code logic is buggy - **Contract bluff** - system advertises capability but rejects it in dispatch - **Structural bluff** - `check_file_exists` passes but doesn't run the test - **Comment bluff** - comment promises behavior code doesn't actually have - **Skip bluff** - `t.Skip("not running yet")` without `SKIP-OK: #<ticket> marker`

Full background: docs/HOST_POWER_MANAGEMENT.md and this Constitution (CONST-035).

CONST-036: Propagation to Submodules

This Constitution, along with CLAUDE.md and AGENTS.md, MUST be propagated to ALL submodules. Each submodule's governance MUST reference this parent Constitution. Changes to this Constitution MUST trigger review of all submodule governance files.

CONST-037: LLMsVerifier Single Source of Truth Mandate

Rule: LLMsVerifier SHALL BE the sole authoritative source for: 1. All model metadata (names, IDs, context windows, capabilities) 2. All provider metadata (endpoints, auth types, supported models) 3. All verification status (verified, partial, failed, pending) 4. All scoring data (overall scores, capability scores, tier rankings) 5. All rate-limit and cooldown state

Prohibition: NO hardcoded model lists, NO hardcoded provider lists, NO simulated model discovery. Any code path that presents a model or provider listing to a user MUST fetch that listing from the LLMsVerifier subsystem or its cached replica.

Anti-Bluff Verification: - The challenge script challenges/scripts/verifier_hardcode_check.sh MUST scan all Go source files for hardcoded model arrays. - Any []string{"gpt-4", "claude-3"} or equivalent literal in production code is a constitutional violation. - The only permitted hardcoded data is the LLMsVerifier service endpoint URL and the list of verification test types.

Enforcement: make test-complete MUST include a test that asserts ModelManager.GetAvailableModels() returns at least as many models as the verifier's database contains for configured providers. A test that passes while the CLI shows a hardcoded list is a TEST BLUFF and violates CONST-035.

CONST-038: Model Provider Anti-Bluff Guarantee

Rule: Every model displayed to an end user MUST have been verified by LLMsVerifier within the last `verification_timeout` period (default: 24h). Models older than this MUST display a “stale” indicator and be deprioritized.

Prohibition Against Test Bluffing: - A unit test that mocks the verifier client and asserts `GetAvailableModels()` returns 3 models DOES NOT satisfy this rule. - An integration test that starts the verifier server, performs real provider discovery, and confirms the model count matches the actual provider API response DOES satisfy this rule. - The Makefile target `make test-verifier-integration` MUST exist and MUST run without mocks.

The “Tests Pass But Features Don’t Work” Guarantee:

NO TEST MAY PASS UNLESS THE FEATURE IT TESTS IS DEMONSTRABLY USABLE BY AN END USER IN THE SAME BUILD.

- If `TestModelList` passes but `helixcode --list-models` shows hardcoded data, the test is a BLUFF.
- If `TestProviderHealth` passes but the health endpoint returns `200 OK` for a provider that is actually down, the test is a BLUFF.
- If `TestLLMGeneration` passes but `--prompt "hello"` returns a simulated string, the test is a BLUFF.
- Bluff tests MUST be rewritten or deleted. There is no “grandfather” exception.

Evidence Standard: Every test that claims to verify model/provider functionality MUST: 1. Call a real API endpoint or a real verifier database 2. Assert on response content that could only come from that real source 3. Include a test that runs the CLI binary with `--list-models` and checks output against verifier data

CONST-039: Real-Time Model Status Accuracy

Rule: Model status (available, rate-limited, cooldown, offline, deprecated) displayed to users MUST reflect the actual state as known by LLMsVerifier within `max_staleness` seconds (default: 60s).

Polling vs. Push: - If WebSocket/SSE push is unavailable, the system MUST poll LLMsVerifier at most every `status_poll_interval` (default: 30s). - The TUI MUST display a “last updated” timestamp with every model listing. - Models in “cooldown” or “rate-limited” state MUST show the estimated recovery time if known.

Accuracy Verification: - Challenge script `challenges/scripts/model_status_accuracy_challenge.sh` MUST: 1. Artificially rate-limit a provider by exhausting its quota 2. Wait for the status to propagate to the verifier 3. Check that `helixcode --list-models` shows the rate-limited status within 60s 4. Check that `SelectOptimalModel()` no longer selects the rate-limited model

Prohibition: Status indicators that are “always green” or that lag >60s behind reality violate this rule.

CONST-040: All Providers and Models Integration Mandate

Rule: HelixCode MUST integrate with ALL providers and models that LLMsVerifier supports, subject only to: 1. The provider being explicitly disabled in configuration (enabled: false) 2. The API key being absent and the provider requiring one 3. The provider being marked deprecated in the verifier database

Minimum Provider Set (SHALL NOT be reduced without constitutional amendment):

Provider	Auth Type	Required Env Var
OpenAI	API Key	OPENAI_API_KEY
Anthropic	API Key / OAuth	ANTHROPIC_API_KEY
Gemini	API Key	GEMINI_API_KEY
DeepSeek	API Key	DEEPSEEK_API_KEY
Groq	API Key	GROQ_API_KEY
Mistral	API Key	MISTRAL_API_KEY
xAI	API Key	XAI_API_KEY
OpenRouter	API Key	OPENROUTER_API_KEY
Ollama	Local	None (auto-detect)
Llama.cpp	Local	None (auto-detect)

Integration Requirement: For every provider in the minimum set: - There MUST be a provider adapter file in `internal/llm/` or `internal/verifier/adapters/` - There MUST be a `*_test.go` file with real API tests (skipped only if `HELIX_SKIP_LIVE_PROVIDER_TESTS` is set) - There MUST be a challenge script in `challenges/scripts/` - The model listing MUST include models from this provider when the provider is enabled

CONST-041: MCP / LSP / ACP / Embedding / RAG / Skills / Plugins Integration Mandate

Rule: LLMsVerifier integration SHALL extend beyond basic model listing to cover ALL capability dimensions:

1. **MCP (Model Context Protocol):** The verifier MUST report which models support MCP tool calling. HelixCode's MCP subsystem MUST consult verifier capability flags before selecting a model for tool-use tasks.
2. **LSP (Language Server Protocol):** The verifier MUST report code-analysis capabilities. Models without `code_analysis` capability MUST NOT be selected for refactoring or debugging tasks.
3. **ACP (Agent Capability Protocol):** The verifier MUST report multi-agent coordination support. Models with `supports_parallel_tool_use` MUST be preferred for ACP workflows.
4. **Embedding:** The verifier MUST report `supports_embeddings` for each model. The CogneeConfig embedding model selection MUST be verifier-aware.
5. **RAG (Retrieval-Augmented Generation):** The verifier MUST report context-window sizes. RAG chunking strategies MUST adapt to the selected model's `context_window_tokens` as reported by the verifier.
6. **Skills / Plugins:** The verifier MUST track plugin compatibility. Models flagged `plugin_compatible` MUST be used when skill/plugin execution is required.

Capability Checklist (MUST be verified by challenge): - [] MCP tool calling verified for at least 3 providers - [] LSP code-analysis verified for at least 3 providers - [] ACP parallel tool use verified for at least 2 providers - [] Embedding generation verified for at least 2 providers - [] RAG context-window adaptation verified - [] Skills/plugin execution verified for at least 2 providers

Prohibition: Capability flags MUST NOT be hardcoded. The `Provider.GetCapabilities()` method MUST return data sourced from the verifier's `VerificationResult` fields.

Article XII — Repository Safety

§12.1 (CONST-042) — No-Secret-Leak

No API key, token, password, certificate, or other credential may be committed to any repository owned by HelixDevelopment or vasic-digital, transitively or otherwise. All secrets live in `.env` files (mode 0600) listed in `.gitignore`. Any leak — to git, logs, build artefacts, screenshots, or external services — is a release blocker until rotated and post-mortemed.

Operational requirements: - Every repo must have `.env`, `.env.local`, `.env.*` (with `!.env.example` exception), `*.pem`, `*.key`, `*.crt`, `id_rsa*` in `.gitignore`. - `scripts/scan-secrets.sh` (or equivalent) must run before every push; failing it blocks the push. - API keys for development are sourced from the canonical `../helix_agent/.env` (mode 0600, never under git) and copied — never symlinked, never committed — into per-repo `.env` files.

Cascade requirement: This article must appear verbatim in every owned-by-us repository's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Owned-by-us repos are listed in `scripts/owned-repos.txt` (or, until that file exists, the meta-repo `propagate-governance.sh` script's submodule walk excluding third-party trees).

§12.2 (CONST-043) — No-Force-Push

No force push, force-with-lease push, history rewrite, branch deletion of `main/master`, or upstream-overwriting operation may be performed without explicit, in-conversation user approval given for that specific operation. Authorization for one push does not extend to subsequent pushes. Bypassing hooks (`--no-verify`), signature verification (`--no-gpg-sign`), or protected-branch rules also requires explicit approval. This applies to every repository in the HelixDevelopment / vasic-digital stack.

Operational requirements: - Local pre-push hook at `scripts/git-hooks/pre-push` (installed by `scripts/install-git-hooks.sh`) must reject `--force` / `--force-with-lease` unless `HELIX_FORCE_PUSH_APPROVED=1` is set. - The hook is a courtesy gate; this constitutional clause is the actual contract. - Regular non-force pushes of new commits to existing branches on already-configured remotes are PERMITTED without per-push approval, scoped to a programme/conversation in which the user has authorised the cadence.

Cascade requirement: Same as §12.1 — verbatim, every owned-by-us repo's three governance files.

- README.md — project overview, quickstart.
- AGENTS.md — guidance for AI coding agents (Codex, Cursor, etc.).
- CLAUDE.md — guidance specifically for Claude Code.
- docs/HOST_POWER_MANAGEMENT.md — CONST-033 background and runbook.

CONST-035 — Anti-Bluff Tests & Challenges (mandatory; inherits from root)

Tests and Challenges in this submodule **MUST** verify the product, not the LLM's mental model of the product. A test that passes when the feature is broken is worse than a missing test — it gives false confidence and lets defects ship to users. Functional probes at the protocol layer are mandatory:

- TCP-open is the FLOOR, not the ceiling. Postgres → execute `SELECT 1`. Redis → `PING` returns `PONG`. ChromaDB → `GET /api/v1/heartbeat` returns 200. MCP server → TCP connect + valid JSON-RPC handshake. HTTP gateway → real request, real response, non-empty body.
- Container Up is NOT application healthy. A `docker/podman ps Up` status only means PID 1 is running; the application may be crash-looping internally.
- No mocks/fakes outside unit tests (already CONST-030; CONST-035 raises the cost of a mock-driven false pass to the same severity as a regression).
- Re-verify after every change. Don't assume a previously-passing test still verifies the same scope after a refactor.
- Verification of CONST-035 itself: deliberately break the feature (e.g. kill <service>, swap a password). The test **MUST** fail. If it still passes, the test is non-conformant and **MUST** be tightened.

CONST-033 clarification — distinguishing host events from sluggishness

Heavy container builds (BuildKit pulling many GB of layers, parallel podman/docker compose-up across many services) can make the host **appear** unresponsive — high load average, slow SSH, watchers timing out. **This is NOT a CONST-033 violation.** Suspend / hibernate / logout are categorically different events. Distinguish via:

- uptime — recent boot? if so, the host actually rebooted.
- loginctl list-sessions — session(s) still active? if yes, no logout.
- journalctl ... | grep -i 'will suspend\|hibernate' — zero broadcasts since the CONST-033 fix means no suspend ever happened.
- dmesg | grep -i 'killed process\|out of memory' — OOM kills are also NOT host-power events; they're memory-pressure-induced and require their own separate fix (lower per-container memory limits, reduce parallelism).

A sluggish host under build pressure recovers when the build finishes; a suspended host requires explicit unsuspend (and CONST-033 should make that impossible by hardening IdleAction=ignore + HandleSuspendKey=ignore + masked sleep.target, suspend.target, hibernate.target, hybrid-sleep.target).

If you observe what looks like a suspend during heavy builds, the correct first action is **not** “edit CONST-033” but bash challenges/scripts/host_no_auto_suspend_challenge.sh to confirm the hardening is intact. If hardening is intact AND no suspend broadcast appears in journal, the perceived event was build-pressure sluggishness, not a power transition.

Article XI §11.9 — Anti-Bluff Forensic Anchor (cascaded from parent CONSTITUTION.md)

Verbatim user mandate (2026-04-29, reasserted multiple times across 2026-05): *“We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”*

Operative rule: **The bar for shipping is not “tests pass” but “users can use the feature.”** Every PASS in this codebase MUST carry positive runtime evidence captured during execution. Metadata-only / configuration-only / absence-of-error / grep-based PASS without runtime evidence are critical defects regardless of how green the summary line looks. No false-success results are tolerable.

This anchor MUST remain in this submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md alongside CONST-047 — see the parent repository’s CONSTITUTION.md for the full text.

CONST-048: Full-Automation-Coverage
Mandate (cascaded from constitution submodule §11.4.25)

> Verbatim user mandate (2026-05-15):
“Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product! This is mandatory constraint which MUST BE respected without ignoring, skipping, slacking or forgetting it!”

No feature / functionality / flow / use case / edge case / service / application on any supported platform of HelixCode may be considered deliverable until covered by automation tests proving six invariants: (1) anti-bluff posture (CONST-035) with captured runtime evidence; (2) proof of working capability end-to-end on target topology (no mocks beyond unit tests — see CONST-050); (3) implementation matches documented promise; (4) no open issues/bugs surfaced — cross-checked against §11.4.15 / §11.4.16 trackers; (5) full documentation in sync per §11.4.12; (6) four-layer test floor per §1 (pre-build + post-build + runtime + paired mutation).

Consuming projects MUST publish a coverage ledger (feature × platform × invariant-1..6 × status) regenerated as part of the release-gate sweep. Gaps tracked per §11.4.15 (UNCONFIRMED: / PENDING_FORENSICS: / OPERATOR-

BLOCKED: with §11.4.21 audit) — rows that quietly omit a platform are CONST-048 violations.

Cascade requirement: This anchor (verbatim or by CONST-048 ID reference) **MUST** appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. See constitution submodule Constitution.md §11.4.25 for the full mandate.

CONST-049: Constitution-Submodule Update Workflow Mandate (cascaded from constitution submodule §11.4.26)

> Verbatim user mandate (2026-05-15):

*“Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we **MUST FIRST** fetch and pull all new changes / work from constitution Submodule first! All changes we apply **MUST BE** committed and pushed to all constitution Submodule upstreams! In case of conflict, **IT MUST BE** carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification **MUST BE** done!”*

Before ANY modification to constitution/Constitution.md, constitution/CLAUDE.md, or constitution/AGENTS.md, the agent or operator **MUST** execute the following 7-step pipeline in order:

1. **Fetch + pull first** inside the constitution submodule worktree — every configured remote fetched, then `git pull --ff-only` (or `--rebase` if non-FF; **NEVER** `--strategy=ours` / `--allow-unrelated-histories` without explicit authorization).
2. **Apply the change** with §11.4.17 classification + verbatim mandate quote.
3. **Validate before commit** — `meta_test_inheritance.sh` (or equivalent), no merge-conflict markers, cross-file consistency.
4. **Commit + push to ALL upstreams** — governance files only (**NEVER** `git add -A`); push to every configured remote. One-upstream commit = CONST-049 violation (also CONST-038/§6.W and §2.1).
5. **Conflict resolution** preserving union of governance content. Force-push to bypass conflicts is **FORBIDDEN** (CONST-043 / §9.2).
6. **Post-**

merge validation — `git submodule update --remote --init` + re-run cascade verifier (CONST-047) confirming the new clause reaches every owned submodule. 7. **Bump consuming project pointer** — `.gitmodules`-tracked submodule pointer advanced to the new constitution HEAD in the SAME commit as cascade work.

Cascade requirement: This anchor (verbatim or by CONST-049 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a force-push without CONST-043 / §9.2 authorization. No escape hatch. See constitution submodule Constitution.md §11.4.26 for the full mandate.

CONST-050: No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (cascaded from constitution submodule §11.4.27)

> Verbatim user mandate (2026-05-15):
“Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interact with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule — <https://github.com/vasic-digital/Challenges>). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule — <https://github.com/HelixDevelopment/HelixQA>). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!).”

Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, TODO, FIXME, “for now”, “in production this would”, or empty-implementation patterns are PERMITTED

only in unit-test sources (*_test.go files invoked without the integration build tag; HelixCode/tests/unit/; etc.). Every other test type — integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA — MUST exercise the real, fully implemented HelixCode system against real infrastructure (real PostgreSQL, real Redis, real LLM endpoints, real containers, real captured devices). Production code (anything under HelixCode/cmd/, HelixCode/applications/, HelixCode/internal/<pkg>/<file>.go not ending *_test.go) MUST NOT import from HelixCode/internal/mocks/.

(B) 100% test-type coverage. HelixCode's codebase MUST be covered by every supported test type the domain warrants: - **Unit** — fast, isolated, mocks permitted per (A). - **Integration** — multi-component, no mocks, real backing services. - **End-to-end (E2E)** — full user-flow exercise on target topology. - **Full automation** — orchestrated suites exercising every feature × platform combination (CONST-048 coverage ledger). - **Security** — authn/authz boundaries, CONST-042 secret-leak scans, input-fuzzing, dependency-CVE scanning, threat-model verification. - **DDoS** — request-flood resilience at advertised throughput tier. - **Scaling** — horizontal + vertical scale behaviour under linear load growth. - **Chaos** — controlled failure injection (network partition, process kill, disk full, clock skew). - **Stress** — sustained load above advertised tier. - **Performance** — latency / throughput / tail-latency invariants vs SLO baselines. - **Benchmarking** — micro + macro suites with historical p95-drift detection. - **UI** — visual-regression + DOM-state + interaction-flow coverage on every target platform's UI surface. - **UX** — flow-correctness + accessibility + i18n + visual-cue ordering (§11.4.23 composition). - **Challenges** — vasic-digital/Challenges submodule (at ./Challenges/) fully incorporated; per-feature Challenge scripts with captured runtime evidence. - **HelixQA** — HelixDevelopment/HelixQA submodule (at ./HelixQA/) fully incorporated; ALL written test banks executed; full autonomous QA sessions run as

part of release gates with captured wire evidence per check.

Required dependency submodules

(recursive per CONST-047): - Challenges — git@github.com:vasic-digital/Challenges.git — incorporated at ./Challenges/. - HelixQA — git@github.com:HelixDevelopment/HelixQA.git — incorporated at ./HelixQA/. - Any additional functionality submodules under vasic-digital/* / HelixDevelopment/* orgs that HelixCode depends on — incorporate rather than duplicate work the orgs already maintain.

Submodule pointers MUST be bumped to upstream HEAD in the SAME commit as any dependent cascade work (CONST-049 step 7). Pointer drift = CONST-050 violation.

Cascade requirement: This anchor (verbatim or by CONST-050 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. See constitution submodule Constitution.md §11.4.27 for the full mandate.

CONST-051: Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (cascaded from constitution submodule §11.4.28)

> Verbatim user mandate (2026-05-15): *“All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory - we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main project's codebase! All on equal basis! Equally important! We MUST take it into the account, analyze it, extend it, create missing tests, do full testing of it, fill the gaps (if any), fix any issues that we discover or they pop-up, write and extend the documentation, user guides, manulas, diagrams, graphs, SQL definitions, Website(s) and all other relevant materials! We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE*

ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE accessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project - directly from project's root project_name/submodule_name or some more proper structure project_name/submodules/submodule_name!"

Three cooperating invariants apply to every HelixCode-owned submodule (those whose upstream origin lives under vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, Boat0S123456, Helix-Flow, Helix-Track, Server-Factory, or any subsequently authorised org):

(A) Equal-codebase. Every owned-by-us submodule is an **equal part** of HelixCode's codebase. The same engineering practice — analysis, extension, test creation, gap-filling, bug-fix, documentation (user manuals, guides, diagrams, graphs, SQL definitions, website pages, all materials) — applies to each owned submodule on equal basis. A round of work that improves only HelixCode's main while leaving an owned-submodule deficiency unaddressed is a CONST-051 violation, severity-equivalent to a §11.4 PASS-bluff at the project-scope layer. The §11.4.25 / CONST-048 coverage ledger MUST list every owned submodule as an in-scope target.

(B) Decoupling / reusability. Owned submodules MUST remain fully decoupled from HelixCode (and any other consuming project). No HelixCode-specific context, hardcoded paths, hostnames, asset names, or runtime assumptions may be introduced into an owned submodule's source tree. When a submodule needs information from HelixCode, the honest path is configuration injection (env var, config file, constructor parameter) — never a hardcoded reach into the parent's tree. Every owned submodule MUST be project-not-aware, fully reusable, modular, and completely testable as a standalone repository.

(C) Dependency-layout. Every dependency that an owned submodule consumes **MUST** be accessible from HelixCode's root at one of two canonical paths: - <repo_root>/<submodule_name>/ (flat layout — current HelixCode layout for Challenges, HelixQA, Containers, Security, etc.) - <repo_root>/submodules/<submodule_name>/ (grouped layout — alternate)

Nested own-org submodule chains are FORBIDDEN. A submodule **MUST NOT** have its own .gitmodules entries pulling in further owned-by-us repos. Every dependency required by submodule X is added to HelixCode's root at the canonical path; X reaches it via documented import / SDK path / runtime resolver — never via its own nested submodule pointer. Third-party submodules (not under our orgs) are exempt — they **MAY** appear at any depth.

The owned-org list is dynamically discoverable at any time via `gh org list / glab CLIs` or the orgs' public APIs.

Cascade requirement: This anchor (verbatim or by CONST-051 ID reference) **MUST** appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the codebase-completeness layer. No escape hatch. See constitution submodule Constitution.md §11.4.28 for the full mandate (audit gates, mutation pairs, workflow integration). >>>>>>
9f5637d2d695cd5fcf8349d1f1b8bf780fa5d865

Amendment Process

Constitution amendments require: 1. Written proposal with rationale 2. Challenge demonstrating the need 3. 72-hour review period 4. Approval by project architect 5. Update to all submodule governance files

This Constitution is the supreme law of the HelixCode project. No code, test, or process may contradict it.

<<<<<< HEAD ## Article XI §11.9 — Anti-Bluff Forensic Anchor (Cascaded)

Verbatim user mandate: “We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This **MUST NOT** be the case and execution of tests and Challenges **MUST** guarantee the quality, the completion and full usability by end users of the product!”

Operative rule: The bar for shipping is not “tests pass” but “users can use the feature.” Every **PASS** in this codebase **MUST** carry positive runtime evidence captured during execution. No false-success results are tolerable.

Bluff Taxonomy (cascaded from root CONSTITUTION.md)

- Wrapper bluff — assertions **PASS** but exit-code logic is buggy
- Contract bluff — advertises capability but rejects in dispatch
- Structural bluff — file exists but doesn’t contain working code
- Comment bluff — comment promises behavior code doesn’t have
- Skip bluff — `t.Skip()` without **SKIP-OK: #** marker

Submodule Decoupling & Reusability — Mandatory

Status: Mandatory. Non-negotiable.

Rule: This repository is a **shared submodule** consumed by multiple independent consumer projects. Its value depends on staying **fully decoupled and reusable**. No change in this repository may introduce coupling that breaks its standalone reusability for any consumer.

Prohibited inside this repository:

1. Hardcoding any specific consumer project’s name, paths, platform list, version strings, release-naming conventions, branding, or feature names.
2. `import` / dependency on any consumer-project namespace, package, or build coordinate.
3. Embedding consumer-project-specific governance, rule numbering, or release cadence into this repository’s `CONSTITUTION.md` / `CLAUDE.md` / `AGENTS.md`.
4. Assuming this repository is consumed by a particular CLI, build system, language toolchain version, or target architecture beyond what its public interface documents.

Required inside this repository:

1. All public surfaces (APIs, CLIs, configuration files, environment variables, scripts) **MUST** be expressed in terms of **THIS** repository’s own domain — not any consumer’s.

2. Governance MUST describe responsibilities and contract from THIS repository's perspective. Consumer projects appear as illustrative examples at most, never as load-bearing requirements.
3. Cross-project rules adopted from a consumer (such as a cross-platform impact mandate) MUST be phrased generically — "every consuming project's full platform matrix" — and never hardcode any single consumer's matrix.

Why: Repositories like this one have shipped changes in the past where one consumer's platform list, feature names, or rule numbering leaked into shared-repo governance — and then collided at merge time with another consumer's parallel work, leaving the repository unmergeable until manual conflict resolution stripped the consumer-specific text back out. Decoupling is the only mechanism that preserves this repository's value as shared infrastructure.

Recursive scope: any submodule this repository consumes inherits the same decoupling+reusability rule. Third-party upstream submodules that this repository merely vendors (e.g. open-source tools under a tools/opensource/ tree, if present) are explicitly out of scope — we are not their owners.

CONST-047 — Recursive Submodule Application Mandate (cascaded from root CONSTITUTION.md)

Verbatim user mandate (2026-05-14): *"Make sure all work we do is applied ALWAYS to all Submodules we control under our organizations (vasic-digital and HelixDevelopment) fully recursively everywhere with full bluff-proofing and comprehensive documentation, user manuals and guides and full tests and Challenges coverage!"*

Every engineering deliverable produced for the main project MUST be applied — fully and recursively — to every owned submodule under the vasic-digital and HelixDevelopment GitHub organizations. Each owned submodule (including this one) MUST receive in lockstep: (1) anti-bluff posture (CONST-035 / Article XI §11.9), (2) comprehensive documentation matching actual capabilities, (3) full tests + Challenges coverage with captured runtime evidence, (4) recursive propagation through nested submodules under the same orgs, (5) synchronized commits when meta-repo state advances this surface.

See the root CONSTITUTION.md §CONST-047 for the full mandate. This anchor MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md.

CONST-048: Full-Automation-Coverage Mandate (cascaded from constitution submodule §11.4.25)

Verbatim user mandate (2026-05-15): "Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product! This is mandatory constraint which MUST BE respected without ignoring, skipping, slacking or forgetting it!"

No feature / functionality / flow / use case / edge case / service / application on any supported platform of this submodule is deliverable until covered by automation tests proving six invariants: (1) anti-bluff posture with captured runtime evidence (CONST-035); (2) proof of working capability end-to-end on target topology; (3) implementation matching documented promise; (4) no open issues/bugs surfaced; (5) full documentation in sync; (6) four-layer test floor (pre-build + post-build + runtime + paired mutation).

Cascade requirement: This anchor (verbatim or by CONST-048 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's CONSTITUTION.md §CONST-048 and constitution submodule Constitution.md §11.4.25 for the full mandate. ##
CONST-049: Constitution-Submodule Update Workflow Mandate (cascaded from constitution submodule §11.4.26)

Verbatim user mandate (2026-05-15): "Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE committed and pushed to all constitution Submodule

upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!"

Before ANY modification to constitution/{Constitution,CLAUDE,AGENTS}.md in the parent project, the agent or operator MUST execute the 7-step pipeline: (1) fetch + pull first inside the constitution submodule worktree; (2) apply the change with §11.4.17 classification + verbatim mandate quote; (3) validate (meta-test + no merge-conflict markers + cross-file consistency); (4) commit + push to EVERY configured upstream of the constitution submodule (governance files only — never git add -A); (5) careful conflict resolution preserving union of governance content (force-push forbidden per CONST-043 / §9.2); (6) post-merge git submodule update --remote --init + re-run cascade verifier (CONST-047); (7) bump consuming project's .gitmodules pointer to the new constitution HEAD in the SAME commit as cascade work.

Cascade requirement: This anchor (verbatim or by CONST-049 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's CONSTITUTION.md §CONST-049 and constitution submodule Constitution.md §11.4.26 for the full mandate. ## CONST-050: No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (cascaded from constitution submodule §11.4.27)

Verbatim user mandate (2026-05-15): "Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interact with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule — <https://github.com/vasic-digital/Challenges>). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule — <https://github.com/HelixDevelopment/HelixQA>). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!)."

Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, TODO, FIXME, “for now”, “in production this would”, or empty-implementation patterns are PERMITTED only in unit-test sources. Every other test type — integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA — MUST exercise this submodule’s real, fully implemented system against real infrastructure. Production code MUST NOT import mock paths.

(B) 100% test-type coverage. Codebase MUST be covered by every supported test type the domain warrants: unit, integration, E2E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges (vasic-digital/Challenges submodule fully incorporated), HelixQA (HelixDevelopment/HelixQA submodule fully incorporated, with full autonomous QA sessions executing every registered test bank with captured wire evidence).

Required dependency submodules (recursive per CONST-047): Challenges + HelixQA + any other functionality submodules under vasic-digital/HelixDevelopment orgs this submodule depends on.

Cascade requirement: This anchor (verbatim or by CONST-050 ID reference) MUST remain in this submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project’s CONSTITUTION.md §CONST-050 and constitution submodule Constitution.md §11.4.27 for the full mandate. ## CONST-051: Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (cascaded from constitution submodule §11.4.28)

Verbatim user mandate (2026-05-15): “All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory - we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project’s codebase! We MUST work on that code as much as we do with main project’s codebase! All on equal basis! Equally important! ... We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE accessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project - directly

from project's root project_name/submodule_name or some more proper structure project_name/submodules/submodule_name!"

Three cooperating invariants apply to every owned-by-us submodule (orgs: vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory, plus any subsequently authorised org — discoverable via `gh org list / glab`):

(A) Equal-codebase. This submodule is an EQUAL part of every consuming project's codebase. The consuming project's engineering practice — analysis, extension, test creation, gap-filling, bug-fix, documentation (user manuals, guides, diagrams, graphs, SQL definitions, website pages, all materials) — applies to this submodule on equal basis. Coverage ledgers (CONST-048) list this submodule as an in-scope target.

(B) Decoupling / reusability. This submodule MUST remain fully decoupled from any specific consuming project. NEVER inject project-specific context (hardcoded paths, hostnames, asset names, naming schemes). Stay project-not-aware, reusable, modular, completely testable as a standalone repository. When parent-project info is needed, use configuration injection (env var, config file, constructor parameter) — never a hardcoded reach.

(C) Dependency-layout. Any dependency this submodule consumes MUST be accessible from the consuming project's root at `<root>/<name>/` or `<root>/submodules/<name>/`. **Nested own-org submodule chains are FORBIDDEN** — this submodule MUST NOT have its own `.gitmodules` entries pulling in further owned-by-us repos. Third-party submodules are exempt.

Cascade requirement: This anchor (verbatim or by CONST-051 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's CONSTITUTION.md §CONST-051 and constitution submodule Constitution.md §11.4.28 for the full mandate.

9f5637d2d695cd5fcf8349d
1f1b8bf780fa5d865 ##
CONST-052: Lowercase-
Snake_Case-Naming
Mandate (cascaded from
constitution submodule
§11.4.29)

Verbatim user mandate (2026-05-15): *"naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MUST HAVE lowercase names with space separator between the words of ' ' character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE renamed! However, since this will most likely break some of the functionalities renaming we do MUST BE applied to all references to particular Submodule or directory! ... There MUST BE reasonable exceptions for this rules - source code for programming languages or Submodules which apply different naming convention - Android, Java, Kotlin and others. ... Upstreams directory which all of our projects and Submodules have MUST BE renamed to the lowercase

letters too, however root project containing the
install_upstreams system command (it is exported in out
paths in our .bashrc or .zshrc) MUST BE updated to fully
work with both Upstreams and upstreams directory. ...
NOTE: Rules lowercase / snake-case do apply to all project
files as well and references to it and from them! “*

<<<<<< HEAD Every directory, submodule, and file in this
submodule MUST use lowercase snake_case names. Existing non-
compliant names MUST be renamed atomically with updates to
every reference (configs, docs, source-code imports, governance
files). Reference drift after rename = CONST-052 violation of
equal severity to the rename itself.

Common-sense exceptions (technology-preserving):

language-mandated case for Java/Kotlin/Android/Apple/C#/Swift
INSIDE language-roots; vendor/upstream third-party submodules
keep upstream names; build artefacts (node_modules,
__pycache__, .git, target, build, bin) keep tool-mandated names.
The test “does renaming break the technology?” trumps the rule.

Upstreams/ → upstreams/ transition: the constitution submodule’s
install_upstreams.sh (exported via .bashrc/.zshrc) supports BOTH
directory layouts; lowercase wins when both present.

Test coverage of renames (per CONST-050(B)): regression test
for reference resolution + full test-type matrix run + anti-bluff
wire-evidence captured.

Cascade requirement: This anchor (verbatim or by CONST-052 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's CONSTITUTION.md §CONST-052 and constitution submodule Constitution.md §11.4.29 for the full mandate.

Every directory, submodule, and file in HelixCode MUST use lowercase snake_case names. Existing non-compliant names (HelixCode/, Challenges/, Containers/, HelixAgent/, HelixQA/, Security/, Github-Pages-Website/, Upstreams/, Dependencies/, etc.) MUST be renamed as part of the phased migration opened by this clause. Every reference (configs, docs, links, source-code imports, governance files) MUST be updated atomically with the rename — reference drift after a rename is a CONST-052 violation of equal severity to the rename itself.

Common-sense exceptions (technology-preserving): language-mandated case for Java/Kotlin/Android/Apple/C#/Swift INSIDE the language root (submodule root follows our convention; subtree follows language convention); vendor/upstream third-party submodules keep upstream names; build artefacts (node_modules, __pycache__, .git, target, build, bin) keep tool-mandated names. The test “does renaming break the technology?” trumps the rule.

Upstreams/ → upstreams/ transition: the constitution submodule's `install_upstreams.sh` (exported via `.bashrc/.zshrc`) supports BOTH `Upstreams/` and `upstreams/` directory layouts (commit 45d3678 of the constitution submodule); lowercase wins when both present.

Test coverage of renames (per CONST-050(B)): every rename batch ships with (i) regression test verifying every reference now resolves, (ii) full test-type matrix run post-rename, (iii) anti-bluff wire-evidence captured.

Phased execution per the operator's explicit instruction: comprehensive brainstorming → phase-divided plan → fine-grained tasks/subtasks → every change covered by every applicable test type. §11.4.20 subagent delegation for cross-cutting rename sweeps.

Cascade requirement: This anchor (verbatim or by CONST-052 ID reference) **MUST** appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the reference-integrity layer. No escape hatch beyond the common-sense exceptions enumerated above. See constitution submodule `Constitution.md` §11.4.29 for the full mandate. >>>>>>
9f5637d2d695cd5fcf8349d1f1b8bf780fa5d865

CONST-053: .gitignore + No-Versioned-Build-Artifacts Mandate (cascaded from constitution submodule §11.4.30)

Verbatim user mandate (2026-05-15): *“every project module, every Submodule, every service and application **MUST HAVE proper .gitignore file!** We **MUST NOT** git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivate which we can recreate by executing proper mechanism for generating **MUST NOT be versioned!** We **MUST** pay attention what is going to be committed every time we are preparing to execute commit! If any violation is detected it **MUST be fixed before commit is executed!**”*

Every project module, owned-by-us submodule, service, and application **MUST** ship a proper `.gitignore`. Forbidden-from-version-control classes:

1. **Build artefacts:** `/bin/`, `/build/`, `/dist/`, `/out/`, `target/`, `*.exe`, `*.dll`, `*.so`, `*.dylib`, `*.a`, `*.o`, `*.class`, `*.pyc`, generator-produced files when the generator is committed.
2. **Cache files:**
`__pycache__/`, `.pytest_cache/`, `.mypy_cache/`, `.ruff_cache/`,

node_modules/, .next/, .cache/, .gradle/, .terraform/, language-server caches.

3. **Temp files:** *.tmp, *.swp, *~, .DS_Store, Thumbs.db, *.orig, *.rej.
4. **Sensitive-data files:** .env, .env.* (allow .env.example placeholder only — no real secrets even as examples), *.pem, *.key, *.crt, id_rsa*, id_ed25519*, .netrc, secrets/, api_keys.sh.
5. **Generated reports/logs:** *.log, coverage.out, htmlcov/, runtime captures unless reference assets.
6. **OS/IDE personal state:** .idea/, .history/, .vscode/ (except shared settings).

Anti-bluff invariant: .gitignore line alone is not sufficient — no file matching the forbidden patterns may be CURRENTLY TRACKED. A tracked *.log despite the ignore-line is a violation of equal severity to no ignore-line at all.

Pre-commit attention: every commit author (human OR agent) MUST inspect `git diff --staged + git status` BEFORE executing the commit. Forbidden-class hits abort the commit until fixed (unstage, add to .gitignore, scrub if already-tracked). Gate CM-GITIGNORE-PRECOMMIT-AUDIT + paired mutation.

Secret-leak intersection (CONST-042 / §11.4.10): a .env leak is BOTH a CONST-053 and a CONST-042 violation; rotation + post-mortem required.

Recreatable-content test: if a documented mechanism regenerates the file from sources, it is a build derivative and MUST be ignored. The committed sources MUST include the generator.

Cascade requirement: This anchor (verbatim or by CONST-053 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. See constitution submodule Constitution.md §11.4.30 for the full mandate.

CONST-054: Submodule-Dependency-Manifest Mandate (cascaded from constitution submodule §11.4.31)

Verbatim user mandate (2026-05-15): *“We MUST HAVE mechanism for each Submodule to determine / know what are its Submodule dependencies so new projects or palces we are incorporate them can add these Submodules to the project root and make them available! Suggested idea is configuration file with expected Submodules Git ssh urls*

perhaps? New project can read it, and recursively add each Submodule to the root of the project and install / expose it to everyone."

Every owned-by-us submodule **MUST** ship helix-deps.yaml at its root declaring its own-org dependencies. Schema: schema_version, deps: [{name, ssh_url, ref, why, layout: flat|grouped}], transitive_handling.{recursive,conflict_resolution}, language_specific_subtree. Tooling: incorporate-submodule <ssh-url> adds the submodule at the parent project's canonical path (CONST-051(C)), reads helix-deps.yaml, recurses for each declared dep, aborts on conflicting refs, emits <root>/.helix-manifest.yaml audit record.

Anti-bluff guarantee: every manifest paired with a Challenge that bootstraps a throwaway consuming project, runs incorporate-submodule, asserts produced layout matches the manifest, runs the submodule's own tests against the bootstrapped layout, captures wire evidence per §11.4.2. A manifest without this proof is a CONST-054 violation.

§11.4.31 / CONST-054 is the **operational complement** of CONST-051(C): nested own-org submodule chains are **FORBIDDEN** — manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root.

Cascade requirement: This anchor (verbatim or by CONST-054 ID reference) **MUST** appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to §11.4 PASS-bluff at the dependency-graph layer. See constitution submodule Constitution.md §11.4.31 for the full mandate.

CONST-055: Post-Constitution-Pull Validation Mandate (cascaded from constitution submodule §11.4.32)

Verbatim user mandate (2026-05-15): "Every time we fetch and pull new changes on constitution Submodule we MUST process the whole project and all Submodule (deep recursively) for validation and verification taht every single rule or mandatory constraint is followed and respected! If it is not, IT MUST BE!"

Whenever a project's constitution submodule is fetched + pulled with any content change, the project **MUST** run scripts/verify-all-constitution-rules.sh **BEFORE** the new constitution HEAD is treated as canonical for any other work. The sweep re-runs the governance-cascade verifier **AND** every implementable rule gate (CONST-053 .gitignore audit, CONST-051(C) nested-own-org-chain audit, CONST-052 case audit, CONST-050(A) mock-from-

production audit, CONST-035 anti-bluff smoke, etc.) against the post-pull tree. Failures populate the project's Issues tracker per §11.4.15 (Status: Reopened, Type: Bug); closure requires positive-evidence per §11.4.

Pull-time invocation: `git submodule update --remote constitution` triggers the sweep automatically (post-update hook OR commit-wrapper invocation). Operator-explicit manual invocation also available.

Anti-bluff: the sweep's own meta-test (paired mutation per §1.1) plants a known violation of each enforced gate and asserts the sweep reports FAIL for the planted gate. A sweep that exits PASS without running every implementable gate is a CONST-055 violation.

CONST-055 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule — without it, new rules cascade as anchors but never get enforced.

Cascade requirement: This anchor (verbatim or by CONST-055 ID reference) **MUST** appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to §11.4 PASS-bluff at the constitutional-enforcement layer. See constitution submodule Constitution.md §11.4.32 for the full mandate.

CONST-056: Mandatory install_upstreams on clone/add Mandate (cascaded from constitution submodule §11.4.36)

Verbatim user mandate (2026-05-15): "Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zhrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository's upstreams!"

Every clone / add of a Git repository under HelixCode **MUST** be followed by `install_upstreams` invocation from the repository's root IF its tree contains `upstreams/` (or legacy `Upstreams/` per CONST-052 transition) populated with `*.sh` recipe files. The utility (installed on operator's PATH via `.bashrc/.zhrc`; implementation in the constitution submodule's `install_upstreams.sh` — already

supports BOTH directory names since constitution commit 45d3678) reads the recipe files, configures every declared upstream as a named git remote, and fans out origin push URLs.

Skipping the invocation when upstreams/ is present silently breaks §2.1 (multi-upstream push is the norm) — the next push lands on only one upstream. Gate CM-INSTALL-UPSTREAMS-ON-CLONE + paired mutation. Automation: the future incorporate-submodule per CONST-054 auto-invokes; manual invocation supported. Pre-commit check: `git remote -v | grep -c push` reports expected count.

Cascade requirement: This anchor (verbatim or by CONST-056 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.36 for the full mandate.

CONST-057: Type-aware Closure-Status Vocabulary (cascaded from constitution submodule §11.4.33)

Every project tracking work items by Type per §11.4.16 MUST close them with the Type-appropriate terminal ****Status:**** value, drawn from this 3-element closed map:

Item **Type:**	Closure **Status:** value
Bug	Fixed (→ Fixed.md)
Feature	Implemented (→ Fixed.md)
Task	Completed (→ Fixed.md)

The (→ Fixed.md) suffix is preserved across all three so the existing migration-discipline tooling (atomic Issues.md → Fixed.md move per §11.4.19) keeps working without per-Type branching. Generators (generate_issues_summary.sh, generate_fixed_summary.sh, the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all “closed, positive evidence captured”) while preserving the literal in the emitted document.

Closing a Feature with Fixed (→ Fixed.md) or a Task with Implemented (→ Fixed.md) is a CONST-057 violation. Gate CM-CLOSURE-VOCAB-TYPE-AWARE walks every Fixed.md heading + every Issues.md heading whose ****Status:**** is one of the three terminal values and asserts the Status-Type match. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.23.

Cascade requirement: This anchor (verbatim or by CONST-057 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.33 for the full mandate.

CONST-058: Reopened-Source Attribution Mandate (cascaded from constitution submodule §11.4.34)

Every Issues.md (or equivalent project tracker) heading whose ****Status:**** is Reopened MUST carry, within 8 non-blank lines of the heading, a ****Reopened-Details:**** line capturing four sub-facts:

- **By:** AI or User (source-of-truth observer who flipped the status). AI covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). User covers operator-side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (YYYY-MM-DD).
- **Reason:** one-line cause classification — chosen from the closed vocabulary { test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered }. Other values permitted with explicit Reason: <free text> annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact justifying the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens without evidence are §11.4.6 / §11.4.7 violations (demotion from Fixed requires captured evidence under the conditions that re-exposed the defect).

The Issues_Summary.md Status column MUST distinguish the four Reopened sub-states by source so a sweep query for “reopens by AI in the last 30 days” is mechanically possible. Suggested column rendering: Reopened (AI: test-failed) vs Reopened (User: manual-testing). Gate CM-ITEM-REOPENED-DETAILS mirrors CM-ITEM-OPERATOR-BLOCKED-DETAILS (§11.4.21 walk pattern). Composes with §11.4.6 / §11.4.7 / §11.4.15 / §11.4.21.

Cascade requirement: This anchor (verbatim or by CONST-058 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.34 for the full mandate.

CONST-059: Canonical-Root Inheritance Clarity (cascaded from constitution submodule §11.4.35)

The **constitution submodule's** three files (constitution/Constitution.md, constitution/CLAUDE.md, constitution/AGENTS.md) ARE the **canonical root** (also called the **parent** files). They contain only universal rules per §11.4.17.

The consuming project's **repository-root files** (<project-root>/CLAUDE.md, <project-root>/AGENTS.md, optionally <project-root>/Constitution.md) are **consumer extensions**. They MUST start with the inheritance pointer (either the Claude-Code native @constitution/CLAUDE.md import or the portable ## INHERITED FROM constitution/CLAUDE.md heading). They contain only project-specific rules per §11.4.17.

When in doubt about which file to edit: universal rule → constitution submodule's file; project-specific rule → consumer's file. Default consumer-side when uncertain (§11.4.17 — narrower scope is cheap to widen).

Terminology: "the parent CLAUDE.md" / "the root Constitution" → constitution-submodule file at constitution/<filename>; "the project CLAUDE.md" / "this project's AGENTS.md" → consumer-side file at <project-root>/<filename>.

No silent demotion or silent promotion. Moving a rule between layers MUST be a visible commit — git mv of a section if it's a clean clone, or explicit
Lifted from <project> to constitution per §11.4.35 / Demoted from constitution to <project> per §11.4.35 commit-message annotation.

Gate CM-CANONICAL-ROOT-CLARITY verifies (a) consumer's CLAUDE.md opens with the inheritance pointer, (b) constitution submodule's three files are present at the expected path, (c) no ## INHERITED FROM block in the constitution submodule's own files (those ARE the source-of-truth, not consumers). Composes with §11.4.17.

Cascade requirement: This anchor (verbatim or by CONST-059 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.35 for the full mandate.

CONST-060: Fetch-before-edit Mandate (cascaded from constitution submodule §11.4.37)

Verbatim user mandate (2026-05-15): *“Make sure that `feedback_fetch_before_edit` memory rule is part of our constitution Submodule - the root Constitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them! Follow the constitution Submodule documentation for details.”*

The FIRST git-touching action of every session, on every consuming project (owned or third-party), MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u}
git submodule foreach --recursive 'git fetch --all --prune --quiet'
```

If HEAD..@{u} is non-empty, integrate the upstream changes BEFORE any local edit. Acting on stale local state produces three failure modes documented in the originating §11.4.37 incident (multi-agent / parallel-session work): (1) **redundant work** — the agent re-does what a parallel session already finished, (2) **false confidence** — completion reports for already-done work, (3) **divergent history** — duplicate sibling commits that double the conflict surface on next push.

Anti-bluff invariant: the fetch+log check MUST produce captured evidence — the actual HEAD..@{u} output, even if empty. Skipping the check on the basis of “I just fetched” or “nothing could have changed in the last N minutes” is a §11.4.6 (no-guessing) violation: the remote state is not knowable without a fetch.

Cascade requirement: This anchor (verbatim or by CONST-060 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to §11.4 PASS-bluff at the parallel-session-coordination layer. See constitution submodule Constitution.md §11.4.37 for the full mandate. <<<<<< HEAD

Anti-Bluff End-User Quality Guarantee (Escalated via HelixConstitution)

Canonical authority: HelixConstitution/Constitution.md §7.1 and §11.4. This section is this submodule’s binding pointer to those universal clauses.

Forensic anchor — verbatim operator mandate (2026-04-28):

“We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules’s Constitution, CLAUDE.MD and AGENTS.MD as well (if not there already)!”

Operative rule: the bar for shipping is **not** “tests pass” but **“users of any consuming project can use the feature.”** Every PASS MUST carry positive runtime evidence captured during execution that the feature actually works end-to-end. Metadata-only PASS, configuration-only PASS, “absence-of-error” PASS, and source-grep-only PASS without runtime evidence are critical defects.

Required minimum evidence per test category:

Category	Minimum evidence
Unit tests	Real inputs + mutation-verified assertions (stub-swap must cause FAIL)
Integration tests	Real subsystems only; mocks only at honest external boundaries
E2E / Challenge tests	Per-test PASS/FAIL lines + log-file artefact path; RUNTIME layer mandatory

Decoupling constraint: this rule applies to every consuming project’s full platform matrix — specific platform names are not hardcoded here.

							9f5637d2d695cd5fcf8349d1f1b8bf780fa5d865
--	--	--	--	--	--	--	--

CONST-061: Pre-Force-Push Merge-First Mandate (cascaded from constitution submodule §11.4.41)

Verbatim user mandate (2026-05-17): “make sure we bring everything from branches to our side before forc push is done! Afer everything is safely and fully merged and all potential conflicts (if any) resolved, then do force push! make sure nothing isnlost, broken or corrupted on bith sides! add these rules in our root Constitution, CLAUDE.MD, AGENTS.MD (constitution Submodule) if itnis not added already! Extremely important rules and mandatory constraints we MUST HAVE and fully respect!”

Any force-push (--force, --force-with-lease, +<ref>, equivalent history-rewrite) authorised under CONST-043 MUST be preceded by a mechanical 4-step merge-first pipeline:

1. **Fetch every remote** — git fetch --all --prune --tags against origin + every upstream; capture output.
2. **Integrate every divergent commit locally** — rebase / merge / operator-confirmed cherry-pick per appropriate strategy for every non-empty HEAD..<<remote>/<branch> range.
3. **Audit the integrated tree** — no conflict markers anywhere (grep -rn '^<<<<<< \|^===== \$\|^>>>>>>' returns empty in governance + source + test files); no file silently dropped; previously-passing tests still pass; captured-evidence artefacts still validate.
4. **Force-push** — only after steps 1-3 produce clean integration evidence: git push --force-with-lease (NEVER --force alone unless authorised per §9.2 sub-clause 6).

Two-gate composition with CONST-043. §11.4.41 does NOT relax CONST-043’s operator-approval requirement — it adds a SECOND mechanical gate. CONST-043 alone authorises a push that loses remote work; §11.4.41 alone risks pushing without operator awareness. Both required.

Three failure modes prevented: (a) remote-side content loss when parallel sessions land work between fetches; (b) stale-state acts when --force-with-lease reads stale local refs without prior fetch; (c) conflict-driven corruption when markers get committed verbatim (observed 2026-05-17 in helix_qa + containers governance files).

Verification artefact: every governed force-push emits a docs/changelogs/<tag>.md “Force-push merge-first audit” section capturing fetch output, per-remote divergence log, integration

strategy, conflict-marker scan, test delta, push output with lease SHA, + CONST-043 authorisation quote. Gate CM-FORCE-PUSH-MERGE-FIRST + paired mutation.

Cascade requirement: This anchor (verbatim or by CONST-061 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the remote-data-integrity layer. See constitution submodule Constitution.md §11.4.41 for the full mandate.

CONST-068: Shell-script target-shell-parseability mandate (cascaded from constitution submodule §11.4.67)

Verbatim user mandate (2026-05-19): *“any issue we spot must be fixed, bash scripts as well if they are broken!” + “Make sure that this is mandatory rule!”*

Verbatim 2026-05-19 operator mandate: *“all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”*

Every committed shell script MUST be parseable by its target interpreter (sh -n for /bin/sh, bash -n for /bin/bash, etc.) AND MUST declare a shebang matching its actual syntax usage. Bash-only constructs (>(...), <(...), [[]], <<<, arrays, \${var^^}, etc.) used in scripts that may be invoked via sh script.sh MUST be wrapped in eval so the parser sees only a string (target shells like mksh parse the entire script before executing — runtime guards cannot save a parse-time rejection). Honest shebangs only: #!/bin/bash only if bash actually expected; #!/bin/sh requires POSIX-clean body. Fix at source per §11.4.1, never at callsites. Composes with §11.4.1 / §11.4.4 / §11.4.6 / §11.4.50 / §11.4.51. Pre-build gate CM-SCRIPT-TARGET-SHELL-PARSEABLE runs sh -n on every in-scope script. No escape hatch — no --skip-parseability-check, --bash-only-script, --runtime-guard-suffices flag.

Cascade requirement: This anchor (verbatim or by CONST-068 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.67 for the full mandate.

§11.4.68 — Positive Sink-Side / Downstream Evidence Mandate (cascaded from constitution submodule §11.4.68)

Verbatim user mandate (2026-05-20): *“We still do not hear any audio played from D3 device! Arvus Web Dashboard when we play music from D3 shows nothing for Codec In Use! This MUST BE investigated and fixed! How come we passed the tests with Arvus validation? What were values for the Codec In Use field? Empty means nothing! This is not working! It MUST BE FIXED, TESTED AND VERIFIED WITH FULL AUTOMATION TESTING ASAP!!!”*

A test that asserts audio or video routing PASS MUST capture and verify **positive sink-side or downstream evidence** — never config-only, never metadata-only, never PCM-open-state-only. At least one of the closed enumeration MUST be captured for every audio/video routing PASS: (1) sink-side codec-state with non-empty Codec-In-Use matching the expected codec regex; (2) strictly-positive PCM frames-written delta from /proc/asound/.../status hw_ptr; (3) ALSA ELD/EDID-Like-Data showing negotiated channel count + format; (4) ffprobe-on-captured-mp4 with non-zero frame count + expected codec/resolution/fps; (5) recording-analyzer event match per §11.4.2/§11.4.5; (6) tinycap RMS amplitude above the line-level floor. Empty / <unreachable> / <N.E.> / <None> placeholders are NOT positive evidence; a missing-but-required sink is OPERATOR-BLOCKED (release-blocker), never SKIP, never PASS. No escape hatch — no --skip-sink-evidence, --allow-empty-codec, --sink-unreachable-is-pass, --metadata-only-suffices flag exists.

Cascade requirement: This anchor (verbatim or by §11.4.68 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the sink-side-evidence layer. **Canonical authority:** constitution submodule Constitution.md §11.4.68 for the full mandate.

§11.4.70 — Subagent-Driven Execution Is The Default (cascaded from constitution submodule §11.4.70)

Verbatim user mandate (2026-05-20): *“Always do if possible Subagent-driven! Add this into our root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md. This should be the default choice ALWAYS!”*

When executing implementation plans (or any task-decomposed execution flow), the **default execution model is subagent-driven** per superpowers:subagent-driven-development. Inline execution is permitted ONLY when (a) the task is trivial AND fits a single sub-300-line edit, OR (b) the operator explicitly requests inline at brainstorm-handoff time. Subagent-driven is the default because it gives isolated context per task, naturally enforces two-stage review, is parallel-PWU compatible (§11.4.58), creates an anti-bluff seam (§11.4), and survives operator absence. No escape hatch — `--inline-execution-required`, `--no-subagents`, `--monolithic-execution` are NOT permitted flags. Skipping subagent-driven for non-trivial work without recorded operator authorisation is itself a §11.4 PASS-bluff.

Cascade requirement: This anchor (verbatim or by §11.4.70 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the execution-model layer. **Canonical authority:** constitution submodule Constitution.md §11.4.70 for the full mandate.

§11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (cascaded from constitution submodule §11.4.71)

Verbatim user mandate (2026-05-20): “before pushing changes to any upstream for any repository - main repo or Submodule, we MUST fetch and pull all changes. Once these are obtained WE MUST investigate what is different compared to head position we were on last time before fetching and pulling new changes! We MUST understand what is done and for what purpose, easpecially how that does affect our project and our System in general! Any mandatory changes or improvements required by fresh changes we just have brought in MUST BE incorporated, covered with all supported types of the tests which will produce as a result of its success execution REAL PROOFS of working for all componetns and functionalities covered and work fully in anti-bluff manner!”

The everyday-push variant of §11.4.41. EVERY push (every repository — main + every submodule) MUST follow the 5-step cycle: (1) fetch all remotes (`git fetch --all --prune --tags`, capture stdout); (2) pull all upstream branches whose tip differs, resolving conflicts per consumer judgment (never auto---ours/--theirs); (3) investigate the diff vs OUR previous HEAD — read EVERY foreign commit's body, understand what/why/how-it-affects-our-system; (4) integrate mandatory changes with

§11.4.4(b) four-layer coverage + §11.4.43 TDD-fix discipline, every PASS carrying §11.4.5 captured-evidence (REAL PROOFS, not metadata-only); (5) only then push, verifying with `git ls-remote post-push`. No escape hatch — no `--skip-fetch`, `--no-investigate`, `--fast-push`, `--trust-upstream` flag.

Cascade requirement: This anchor (verbatim or by §11.4.71 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the push-discipline layer. **Canonical authority:** constitution submodule Constitution.md §11.4.71 for the full mandate.

§11.4.72 — Audio Top-Priority Mandate (cascaded from constitution submodule §11.4.72)

Verbatim user mandate (2026-05-20): *“Make sure all fixes for audio are always top priority in main working stream!”*

The conductor (main working stream — Claude Code session, AI agent, or human operator) MUST treat audio fixes as the highest-priority class on the serial dispatch queue. Any time the conductor faces a choice between dispatching an audio task vs a non-audio task on the SAME serial resource, the audio task wins. Parallel BACKGROUND subagents (research, refactors, infrastructure documentation) MAY run concurrently with audio work but do NOT preempt audio on the main-stream serial dispatch queue. No escape hatch — there is no “but this non-audio task is faster” or “but this research is more interesting” override; audio-stack regressions are user-perceptible and high-impact while research and refactors can wait.

Cascade requirement: This anchor (verbatim or by §11.4.72 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a process violation at the dispatch-priority layer. **Canonical authority:** constitution submodule Constitution.md §11.4.72 for the full mandate.

§11.4.73 — Main-Specification Document Versioning + Revision Discipline (cascaded from constitution submodule §11.4.73)

Verbatim user mandate (2026-05-20): *“Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Add this into root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md as mandatory rule / constraint applicable ONLY IF we have something like the main specification document or we do recognize something like the main specification document. Document MUST BE updated ALWAYS to follow the versioning rules we are applying here + revision and other properties we have!”*

Applies **only when a project recognises a main specification document**. When it does: (1) every additive operator requirement, refinement, or accepted recommendation MUST be applied to the spec before or as part of the implementing work; (2) spec versioning has two axes — *primary* (V1/V2/V3, bumped for major rewrites by explicit operator decision, old versions archived) and *secondary* (the §11.4.61 metadata-table Revision integer, bumped for every other change); (3) the metadata table MUST stay current (Revision, Last modified, Status summary, Fixed); (4) propagated copies of the rule MUST reference the active specification.V<primary>.md, not a stale archive; (5) on primary bump the old file moves to <spec-dir>/archive/ with Status: superseded. Classification: universal, applicable conditionally per the scope condition.

Cascade requirement: This anchor (verbatim or by §11.4.73 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a release blocker when a project has a main spec and lets it drift.
Canonical authority: constitution submodule Constitution.md §11.4.73 for the full mandate.

§11.4.74 — Submodule-Catalogue-First Discovery + Extend-Don't-Reimplement (cascaded from constitution submodule §11.4.74)

Verbatim user mandate (2026-05-20): *“We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in vasic-digital and HelixDevelopment organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times if they are already done as some of existing universal, reusable general development purpose Submodules! For any missing features that some Submodules we incorporate may be missing we MUST IMPLEMENT the properly and extend those Submodules further! We do control all of the and we CAN and MUST maintain and extend the regularly! All development cycle rules we have MUST BE applied to them and fully respected!”*

Before scaffolding ANY new module, package, helper, or utility, the contributor (human or AI agent) MUST: (1) survey the canonical Submodule catalogue — vasic-digital and HelixDevelopment on both GitHub AND GitLab; (2) inventory existing Submodules; (3) reuse before reimplement — if a Submodule provides the functionality (or 80%+ of it), add it as a Git submodule rather than write fresh; (4) extend in-place when 80%+ matches but features are missing — add the missing features TO THAT SUBMODULE (PR upstream + bump pointer), never as a duplicating consuming-project helper; (5) apply all development-cycle rules to those Submodules; (6) document the survey result in the feature's tracker entry with a Catalogue-Check: field (reuse <org/repo>@<sha> / extend <org/repo>@<sha> / no-match <date>). Classification: universal.

Cascade requirement: This anchor (verbatim or by §11.4.74 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a process violation; duplicate implementations landed without catalogue check are release blockers. **Canonical authority:** constitution submodule Constitution.md §11.4.74 for the full mandate.

§11.4.69 — Universal Sink-Side Positive-Evidence Taxonomy + Mechanical Enforcement (cascaded from constitution submodule §11.4.69)

Verbatim user mandate (2026-05-20): *“THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! Not just for audio but for every single piece of the System!!! Proper full automation when executed with success MUST MEAN that manual testing will be as much positive at least regarding the success results! ... Solution MUST BE universal, generic that solves working flows for all System components and for all future and all existing projects! ... Everything we do MUST BE validated and verified with rock-solid proofs and anti-bluff policy enforcement and fulfillment!”*

Universal generalisation of §11.4.68 (audio-specific) across every user-visible feature class. Every user-visible feature MUST map to one entry in the closed-set §11.4.69 sink-side evidence taxonomy (audio_output, audio_input, video_display, network_throughput, network_connectivity, bluetooth_a2dp, bluetooth_pair, touch_input, sensor, gpu_render, storage_read, storage_write, mediacodec_decode, mediacodec_encode, miracast, cast, boot_service, package_install, permission_grant, wifi_link, wifi_throughput, ethernet_link, display_topology, drm_playback, subtitle_render — open to additions, never contraction). Every PASS for a feature in the taxonomy MUST cite a captured-evidence artefact path matching the required evidence shape. New helper contracts (additive during grace, mandatory after 2026-06-19): ab_pass_with_evidence <description> <evidence_path> (verifies path exists + non-empty), ab_skip_with_reason <description> <closed-set-reason> (reasons: geo_restricted, operator_attended, hardware_not_present, topology_unsupported, network_unreachable_external, feature_disabled_by_config; forbids network_unreachable_external for any taxonomy feature with a sink-side probe); bare ab_pass deprecated (WARN pre-grace, FAIL post-grace). Three pre-build gates + paired §1.1 mutations: CM-SINK-EVIDENCE-PER-FEATURE, CM-NO-FAIL-OPEN-SKIP, CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE. No escape hatch — no --skip-evidence, --config-only-pass, --allow-fail-open-skip, --legacy-ab-pass-permitted flag.

Cascade requirement: This anchor (verbatim or by §11.4.69 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-69-PROPAGATION enforces the anchor literal across the consumer fleet; paired mutation strips the literal → gate FAILs.

Severity-equivalent to a §11.4 PASS-bluff at the sink-side-evidence layer. **Canonical authority:** constitution submodule Constitution.md §11.4.69 for the full mandate.

§11.4.75 — Mechanical Enforcement Without Exception (cascaded from constitution submodule §11.4.75)

Verbatim user mandate (2026-05-20): *“Why do these violations still happen!? This is a serious problem! We cannot rely on stability nor consistency if we cannot respect our Constitution, mandatory rules and constraints! Is there a way to make this always respected, followed and applied without exception fully and unconditionally!? WE MUST HAVE THIS WORKING FLAWLESSLY!!! Do investigate the root causes of such problems! Once all problems are identified WE MUST apply proper mechanisms for this not to happen NEVER EVER AGAIN!”*

The §11.4 covenant historically relied on agent + operator vigilance; three 2026-05-19→20 forensic incidents proved that late-binding enforcement fires hours-to-days after the violator commit reaches every remote. §11.4.75 closes the gap with FIVE independent mechanical enforcement layers — bypassing any single layer does not bypass the discipline: (1) local pre-commit git hook (refuses staged .md lacking sibling .html+.pdf); (2) commit_all.sh integration (_constitution_sibling_check + auto-sync_all_markdown_exports.sh self-repair); (3) local pre-push git hook (re-runs siblings + propagation-gate subset); (4) post-commit auto-repair hook (auto-generates orphan-.md siblings, idempotent + recursion-guarded); (5) local-only final-gate ritual (remote CI DISABLED per User mandate — operator runs pre_build_verification.sh + meta-test before every tag per §11.4.40). Helper contracts: scripts/install_git_hooks.sh, scripts/git_hooks/{pre-commit,pre-push,post-commit,commit-msg}, _constitution_sibling_check. The commit-msg hook enforces a Bypass-rationale: <reason> footer when --no-verify is detected; docs/audit/bypass_events.md accumulates the audit trail. Five gates with paired §1.1 mutations: CM-COVENANT-114-75-PROPAGATION, CM-GIT-HOOKS-INSTALL-SCRIPT, CM-GIT-HOOKS-SOURCE-DIR, CM-COMMIT-ALL-SIBLING-CHECK, CM-CI-WORKFLOW-PRESENT. No escape hatch — no --skip-hooks, --bypass-enforcement, --allow-orphan-md, --ci-not-applicable, --mechanical-enforcement-not-needed flag.

Cascade requirement: This anchor (verbatim or by §11.4.75 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-75-PROPAGATION; paired mutation strips the literal →

gate FAILs. Severity-equivalent to a §11.4 PASS-bluff at the enforcement layer. **Canonical authority:** constitution submodule Constitution.md §11.4.75 for the full mandate.

§11.4.76 — Containers-Submodule Mandate (cascaded from constitution submodule §11.4.76)

Verbatim user mandate (2026-05-20): *“For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemu / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly: <https://github.com/vasic-digital/containers> ([git@github.com:vasic-digital/containers.git](https://github.com/vasic-digital/containers.git)). Containers Submodule contains all means for us to Containerize our code and services! If any feature or Containing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! No bluff work is allowed of any kind!”*

For ANY containerized workload (Docker / Podman / Qemu / Kubernetes / container-backed emulators), every consuming project MUST: (1) install vasic-digital/containers (digital.vasic.containers) as a Git submodule; (2) consume via replace directive during development + pinned commit SHAs in production; (3) boot infra on-demand via pkg/boot + pkg/compose + pkg/health so operators are never required to start podman machine / docker compose up manually — the boot is part of the test entry point (the on-demand-infra invariant); (4) extend the Submodule (PR upstream) for missing runtimes / lifecycle primitives — never reimplement in-project (per §11.4.74); (5) anti-bluff: integration tests claiming to exercise containerized components MUST actually boot them via the Submodule — short-circuit fakes that bypass boot are a §11.4 violation. Tracker rows touching containerization MUST record Catalogue-Check: extend vasic-digital/containers@<sha> (or reuse). Planned gate CM-CONTAINERS-USED scans container-touching PRs for digital.vasic.containers/... imports; paired mutation strips the import + asserts FAIL.

Cascade requirement: This anchor (verbatim or by §11.4.76 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-76-PROPAGATION; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule Constitution.md §11.4.76 for the full mandate.

§11.4.77 — Regeneration-Mechanism-Required Mandate (cascaded from constitution submodule §11.4.77)

Verbatim user mandate (2026-05-20): *“We must be sure that after excluding anything from Git versioning we still have the mechanism which will out of the box obtain or re-generate missing content!”*

Every .gitignore entry excluding (a) >~100 MiB OR (b) any artefact essential to building / running / testing the project MUST carry a documented + automated mechanism to either re-obtain (download from authoritative source: vendor tarball, SDK installer, npm/pip/cargo/go-mod/container registry, dedicated git submodule, S3/GCS) OR re-generate (run from tracked source via build pipeline, code-gen, asset render, captured-evidence replay, container build). Required artefacts per qualifying entry: (1) .gitignore-meta/<entry-slug>.yaml declaring pattern + mechanism-type + script-path + expected-disk-usage + vendor-url-or-source + integrity hash + requires-network + requires-credentials; (2) a non-interactive entry in scripts/setup.sh post-clone bootstrap; (3) a pre-build gate verifying regenerated content present OR a recent .gitignore-meta/.regenerated/<slug>.ok stamp; (4) README + docs/guides/*.md describing the mechanism + manual fallback + time/disk budget + §11.4.10 credentials. Bare .gitignore additions without the mechanism are a §11.4 PASS-bluff variant — codebase appears complete but a fresh clone cannot build/run. No escape hatch — no --skip-regen-mechanism, --gitignore-is-enough, --operator-already-has-content flag. Planned gate CM-GITIGNORE-REGEN-MECHANISM + paired §1.1 mutation (strip a required YAML key → gate FAILs).

Cascade requirement: This anchor (verbatim or by §11.4.77 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-77-PROPAGATION; paired mutation strips the literal → gate FAILs. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. **Canonical authority:** constitution submodule Constitution.md §11.4.77 for the full mandate.

§11.4.78 — CodeGraph Code-Intelligence Mandate (cascaded from constitution submodule §11.4.78)

Verbatim user mandate (2026-05-20): “Make codegraph MANDATORY CHOICE for this purpose for all of our project ... All project which do not have configured and installed codegraph yet MUST DO IT and MUST USE IT!”

Every consuming project worked on by AI coding agents MUST install, initialize, and use **CodeGraph** (<https://github.com/colbymchenry/codegraph>, npm @colbymchenry/codegraph) — a local SQLite semantic code-knowledge-graph exposed to agents over MCP (100% local, no cloud). (1) Install globally via npm with a user-writable npm prefix (no sudo). (2) codegraph init + codegraph index: .codegraph/config.json is tracked, .codegraph/codegraph.db is gitignored with codegraph index as its §11.4.77 regeneration mechanism; the config.json exclude list MUST exclude every credential/secret path per §11.4.10. (3) Wire codegraph serve --mcp into every CLI agent (Claude Code .mcp.json, OpenCode opencode.json, Qwen Code .qwen/settings.json, Crush .crush.json, host-local otherwise) referencing the bare codegraph command on PATH (no hardcoded host path). (4) Cover the integration with an anti-bluff suite whose per-agent end-to-end layer uses an unforgeable challenge (a fact obtainable only by calling a CodeGraph MCP tool, e.g. index node count via codegraph_status); a genuinely un-drivable agent is a documented SKIP per §11.4.3, never a faked PASS. (5) Document in docs/CODEGRAPH.md, kept in sync per §11.4.12 / §11.4.65. CodeGraph is consumed as the published npm package (§11.4.74) — not a git submodule, adds no Git remote. Planned gate CM-CODEGRAPH-WIRED + paired §1.1 mutation (strip a secret-exclusion → gate FAILs).

Cascade requirement: This anchor (verbatim or by §11.4.78 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-78-PROPAGATION; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule Constitution.md §11.4.78 for the full mandate.

§11.4.79 — Own-Org Submodules MUST Be Included in the CodeGraph Index (cascaded from constitution submodule §11.4.79)

Verbatim user mandate (2026-05-21): *“All Submodules we use in the project and that are part of organizations to which we have the full access via GitHub, GitLab and other CLIs MUST BE included into the codegraph database and initialized / scanned / synced!”*

Refines §11.4.78’s exclude-list with a per-submodule-ownership split: (a) own-org submodules (full write access via the project’s CLIs — canonical orgs vasic-digital + HelixDevelopment) MUST be INCLUDED in the index; (b) third-party submodules (the §11.4.74 no-match → vendor path) MUST be EXCLUDED. Operational steps: (1) `git submodule update --remote --merge` to pull latest before re-indexing, respecting load-bearing pins on third-party submodules; (2) adjust `.codegraph/config.json` exclude list to keep own-org paths in scope; (3) re-index via `scripts/codegraph_setup.sh`; (4) verify via `scripts/codegraph_validate.sh` with ≥ 1 probe resolving a symbol living ONLY inside an own-org submodule; (5) paired §1.1 mutation — temporarily add the own-org submodule to exclude → validate MUST FAIL on the cross-submodule probe → restore. An index that lies about reachable symbols is a PASS-bluff against AI agents. Own-org submodules silently excluded without an audit trail in `.codegraph/config.json` comments is a release blocker.

Cascade requirement: This anchor (verbatim or by §11.4.79 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-79-PROPAGATION; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule Constitution.md §11.4.79 for the full mandate.

§11.4.80 — CodeGraph Regular-Update + Sync Automation Mandate (cascaded from constitution submodule §11.4.80)

Verbatim user mandate (2026-05-21): *“We MUST regularly check for the updates and execute codegraph npm updates so the latest version of it is always installed on the host machine! ... Make sure we have proper full automation bash scripts which will run regularly and that these are part of the constitution Submodule ... Make sure all updates, sync processes we do and important codegraph related events are*

all documented under docs/codegraph in Status and Status_Summary documents ... and regularly export them like all other Status docs into the PDF and HTML!"

Three deliverables (all living in the constitution submodule, inherited by reference per §3 — consuming projects invoke at \$ {CONST_DIR}/scripts/codegraph_*.sh, never copy): (1) scripts/codegraph_update.sh — npm-installs latest @colbymchenry/codegraph after a registry version check; appends old/new version to docs/codegraph/Status.md; anti-bluff verifies codegraph --version reflects the new version after install (npm exit 0 ≠ working binary). (2) scripts/codegraph_sync.sh — after a successful update runs codegraph status → codegraph sync . → codegraph status → the project's scripts/codegraph_validate.sh; appends every step's output to BOTH the project's and the constitution's docs/codegraph/Status.md. (3) docs/codegraph/Status.md + Status_Summary.md append-only ledgers, exported to .html + .pdf per §11.4.65. Cadence: weekly floor (per §11.4.45). A consuming project that has not run codegraph_update.sh in >2 weeks AND has open AI-agent work is a release blocker. Paired §1.1 mutation: downgrade installed version → script detects drift → restore.

Cascade requirement: This anchor (verbatim or by §11.4.80 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-80-PROPAGATION; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule Constitution.md §11.4.80 for the full mandate.

§11.4.81 — Cross-Platform-Parity Mandate (cascaded from constitution submodule §11.4.81)

Verbatim user mandate (2026-05-21): "Any Linux-only blocker / issue we have MUST BE created macOS and other supported platforms equivalent! So, depending on platform proper implementation will be used for particular OS! EVERYTHING MUST BE PROPERLY EXTENDED AND UPDATED!"

Every consuming project whose supported-platforms manifest lists more than one OS MUST, for every feature/test/gate/challenge/mutation depending on platform-specific primitives, ship a per-OS-equivalent implementation chosen at runtime via uname -s (or equivalent detection). Three sub-mandates: **(A) Per-OS implementation REQUIRED** — Linux cgroup/systemd//proc primitives MUST have documented per-OS equivalents (POSIX setrlimit/ulimit, macOS launchd, BSD rctl, Windows Job Object) chosen via runtime dispatch. **(B) Per-OS tests REQUIRED** — every platform-dependent gate test MUST have

case "\$(uname -s)" in branches with positive captured evidence per §11.4.2 + §11.4.5 in each branch; SKIP-with-reason acceptable ONLY when the platform genuinely cannot enforce the invariant.

(C) Honest kernel-gap citation + adjacent equivalent test REQUIRED — where a Linux primitive has NO equivalent due to a documented kernel limitation (canonical: XNU does not enforce RLIMIT_AS for unprivileged processes), the test MUST detect the gap at runtime, SKIP with exact kernel reason + reproducer + honest-gap-doc link, AND provide an ADJACENT test exercising the closest invariant the platform CAN enforce (e.g. RLIMIT_CPU+SIGXCPU as the macOS proxy), itself anti-bluff with a paired §1.1 mutation. Gate CM-CROSS-PLATFORM-PARITY scans for case "\$(uname -s)" blocks asserting a non-SKIP branch (or honest-gap citation) per platform in the manifest; paired mutation strips a Darwin branch → gate FAILs. No escape hatch.

Cascade requirement: This anchor (verbatim or by §11.4.81 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-81-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker on multi-platform projects.

Canonical authority: constitution submodule Constitution.md §11.4.81 for the full mandate.

§11.4.82 — Iteration-Speedup Discipline Mandate (cascaded from constitution submodule §11.4.82)

Verbatim user mandate (2026-05-22): *"How can we speed-up this whole development and fixing process? ... Do not forget to all speed optimizations critical rules and mandatory constraints MUST BE all added into our root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md and QWEN.md and all other relevant constitution Submodules files!"*

Iteration cycle time is a first-order quality enabler. Every consuming project's build / test / commit / debug pipeline MUST adopt these speedup disciplines AS MANDATORY (each independently enforceable): (A) Phase-1 forensic (superpowers:systematic-debugging) before any speculative source patch — speculative patches without FACT-grade root cause are §11.4.6 + §11.4.82 violations; (B) Live-ADB-First (or live-equivalent) before any rebuild — strengthens §11.4.51 to a release-blocker mandate; (C) 30-second pre-flight before launching rebuild orchestrators (device/sink reachability, host memory/disk, no stale locks, no orphan processes); (D) persistent build caches outside containers (ccache/sccache/Gradle daemon bind-mounted to host); (E) module-only rebuild for loadable-module-only changes; (F) parallel multi-device testing with

separate qa-results/<TS>/<device-tag>/ outputs; (G) subagent scope discipline + worktree isolation (≤30 min budget, single-responsibility, isolation: "worktree" default); (H) lock-file + stale-process hygiene (clean .git/index.lock, disable auto git-gc in concurrent repos); (I) cycle telemetry per §11.4.24 (commit hash, per-phase wall-clock, speedup-flag set, outcome — aggregated weekly). Gate CM-ITERATION-SPEEDUP-DISCIPLINE audits recent cycles for telemetry citing which of (A)-(I) applied; paired §1.1 mutation strips the speedup-flag column → gate FAILs. No escape hatch — no --skip-phase1-forensic, --no-pre-flight, --rebuild-everything-always, --unlimited-subagent-scope, --ignore-locks, --no-telemetry flag.

Cascade requirement: This anchor (verbatim or by §11.4.82 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-82-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.82 for the full mandate.

§11.4.83 — docs/qa/ End-User Evidence Mandate (cascaded from constitution submodule §11.4.83)

Verbatim user mandate (2026-05-22): "every feature that ships MUST carry a recorded e2e communication transcript + any attached materials under docs/qa/<run-id>/ (per-feature subdirectories). A feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation MUST preserve full bidirectional communication threads as proof."

Every feature that ships MUST carry a recorded end-to-end communication transcript plus any attached materials (screenshots, request/response payloads, audio, file uploads) committed under docs/qa/<run-id>/ — one directory per feature run. Operative rule: (1) every consuming project MUST maintain a docs/qa/ tree, each new feature under docs/qa/<run-id>/ where <run-id> is monotonic + greppable (timestamp / ATM-NNN / other workable-item ID per §11.4.54); (2) transcripts MUST be full bidirectional — every prompt/command sent + every response received (one-sided is not a transcript); (3) attached materials MUST be committed in-repo (no external-only links — that is a §11.4.13 sink-side violation); (4) bot-driven / agent-driven QA automation MUST preserve the full conversation thread as the proof artefact; (5) release gates MUST refuse to tag a version that has any feature-shipping commit without its matching docs/qa/

<run-id>/ directory. A feature with no QA transcript is a §11.4 / §107 PASS-bluff. Composes with §11.4.2 / §11.4.5 / §11.4.13 / §11.4.65 / §11.4.69 / §1.1.

Cascade requirement: This anchor (verbatim or by §11.4.83 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-83-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker — no --qa-evidence-optional escape hatch. **Canonical authority:** constitution submodule Constitution.md §11.4.83 for the full mandate.

§11.4.84 — Working-Tree Quiescence Rule for Subagent Commits (cascaded from constitution submodule §11.4.84)

Verbatim user mandate (2026-05-22): “no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before git add, the committing agent MUST grep its own working tree for mutation markers (MUTATED for paired, // always pass, return json.Marshal shortcut paths, etc.). Any unexplained file in the staging area triggers ABORT.”

No subagent (or main-thread) commit may proceed while any concurrent mutation gate, paired-mutation experiment, or other in-flight mutation is live in the same checkout. Before git add, the committing agent MUST grep its own working tree for mutation markers (MUTATED for paired, // always pass, return json.Marshal shortcut paths, // MUTATION / # MUTATION annotations, _mutated_* filename suffixes, etc.) and explicitly account for every modified file in the staging area; any unexplained file → ABORT. (Forensic case: a logo-fix subagent's git add swept an // always pass JWT-verify mutation residue into an unrelated commit pushed to all four mirrors — a real security-defect window.) Operative rule: (1) pre-git add greps for mutation markers + cross-checks git status --porcelain against the subagent's declared scope; unaccounted entries → ABORT; (2) any active mutation gate MUST be serialised (mutate → assert FAIL → restore → assert PASS) and the working tree verifiably clean before any unrelated commit; (3) concurrent subagents in the SAME checkout MUST coordinate through a lockfile (.git/MUTATION_IN_PROGRESS) — cleaner solution is git worktree add per subagent (composes with §11.4.20/§11.4.70); (4) post-commit mutation-residue-scanner MUST run before push — any commit containing a mutation marker → push BLOCKED.

Cascade requirement: This anchor (verbatim or by §11.4.84 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-84-PROPAGATION; paired mutation strips the literal →

gate FAILs. A mutation marker that lands in a tagged commit is a critical defect regardless of how briefly it persisted. **Canonical authority:** constitution submodule Constitution.md §11.4.84 for the full mandate.

§11.4.85 — Stress + Chaos Test Mandate (cascaded from constitution submodule §11.4.85)

Verbatim user mandate (2026-05-24): *“Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!”*

Every fix or improvement landed MUST ship with full-automation **stress** AND **chaos** test suites exercising edge cases, sustained load, concurrent contention, and failure-injection. Happy-path coverage alone is a §11.4 / §107 PASS-bluff at the resilience layer. **Stress** (closed-set): sustained load ($N \geq 100$ iterations OR ≥ 30 s wall-clock, p50/p95/p99 latency recorded) + concurrent contention ($N \geq 10$ parallel invocations, no deadlock/leak) + boundary conditions (empty/max/off-by-one, each categorised). **Chaos** (closed-set, per fix-class appropriateness): process-death injection + network-fault injection (drop/delay/reorder) + input-corruption injection + resource-exhaustion injection (disk full, OOM, FD exhaustion — refuse cleanly OR degrade, NEVER crash) + state-corruption injection (mid-flight lock loss, partial-write). Every stress + chaos PASS MUST cite a captured-evidence artefact path per §11.4.5 + §11.4.69. Helper library `stress_chaos.sh` provides `ab_stress_run`, `ab_stress_concurrent`, `ab_chaos_kill_pid_during`, `ab_chaos_drop_network_during`, `ab_chaos_corrupt_file_during`, `ab_chaos_oom_pressure_during`, `ab_chaos_disk_full_during`, each composing with `ab_pass_with_evidence` / `ab_skip_with_reason`. Cleanup non-negotiable in trap `'...'` EXIT (cleanup failure = §11.4.14 violation). Four-layer coverage per §11.4.4(b) + paired §1.1 mutation (strip chaos-injection or evidence-capture → gate FAILs). No escape hatch — no `--skip-stress`, `--no-chaos`, `--happy-path-suffices`, `--stress-test-later` flag.

Cascade requirement: This anchor (verbatim or by §11.4.85 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-85-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.85 for the full mandate.

§11.4.86 — Roster/Corpus-Backed Status-Doc Auto-Sync Mandate (cascaded from constitution submodule §11.4.86)

Verbatim user mandate (2026-05-25): *“Make sure that assets and players Status docs are ALWAYS regularly updated and in sync like all others Status docs — any time we add or modify the assets content(s) or we change or add new / remove existing pre-installed video and audio player apps! This MUST WORK OUT OF THE BOX!”*

Some Status docs (§11.4.45) are backed by a tracked roster (installed apps/components) or a tracked asset corpus (test/media asset directory) rather than narrative alone. Their freshness MUST NOT depend on operator vigilance — the moment a roster/corpus member changes (app added/removed/renamed; asset added/modified/removed) the Status doc + Status_Summary + HTML + PDF MUST resync out of the box, mechanically. Mechanism (all must hold): (1) drift-proof fingerprint — sha256 of the sorted member list (NOT mtime), persisted in a sidecar beside the Status doc; (2) a sync helper that regenerates the fingerprint + re-exports HTML+PDF via the §11.4.65 exporter, wired so sync is automatic; (3) a pre-build gate that FAILs when the live fingerprint differs from the persisted one (mirrors §11.4.12 CM-ISSUES-SUMMARY-SYNC + §11.4.45 sync_integration_status); (4) a paired §1.1 mutation corrupting the fingerprint and asserting the gate FAILs. Classification: universal — the consuming project supplies the specific docs, roster/corpus sources, helper, and gate name per §11.4.35.

Cascade requirement: This anchor (verbatim or by §11.4.86 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-86-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker — no --skip-roster-sync, --allow-status-drift, --roster-sync-not-applicable flag. **Canonical authority:** constitution submodule Constitution.md §11.4.86 for the full mandate.

§11.4.87 — Endless-Loop Autonomous Work + Zero-Idle Agent Dispatch + Anti-Bluff Testing Mandate (cascaded from constitution submodule §11.4.87)

Verbatim user mandate (2026-05-26): “*continue in endless loop fully autonomously*” (and any semantically-equivalent phrasing).

When the operator instructs an AI agent to continue in an endless autonomous loop, the agent **MUST** treat it as a **HARD-CONTRACT** covenant: (A) continue working until docs/Issues.md Status-column has zero non-terminal entries AND docs/CONTINUATION.md §3 Active work is empty AND no background subagent is mid-execution AND no external dependency is in-flight; (B) dispatch background subagents for parallelisable work — main + every subagent operate concurrently, “waiting for results” is the **ONLY** acceptable idle reason; (C) every closure lands four-layer test coverage per §11.4.4(b) with captured-evidence (audio/video/network/UI/sysfs physical proofs); (D) the §11.4 anti-bluff covenant family (§11.4.1 / §11.4.2 / §11.4.6 / §11.4.7 / §11.4.27 / §11.4.50 / §11.4.52 / §11.4.68 / §11.4.69 / §11.4.83) is the operative truth-discipline — tests AND HelixQA Challenges bound equally; (E) the loop terminates **ONLY** on all-conditions-met, explicit operator STOP, host-session-safety demand, or scheduled wake on a known-future-actionable signal. No escape hatch — no --idle-OK, --skip-endless-loop, --bluff-permitted-for-this-task, --metadata-only-test-suffices, --no-physical-proof-required flag.

Cascade requirement: This anchor (verbatim or by §11.4.87 reference) **MUST** appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-87-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.87 for the full mandate.

§11.4.88 — Background-Push Mandate: Commit-Lock Release Immediately After Commit, Push Runs Detached (cascaded from constitution submodule §11.4.88)

Forensic anchor (2026-05-26): a single commit_all.sh held its flock ~5 hours because do_push ran synchronously after the commit landed — every subsequent commit blocked on a slow mirror push irrelevant to the local commit’s durability. Implementation seam

for §11.4.87(B) zero-idle. The mandate: (A) `.git/.commit_all.lock` MUST be released IMMEDIATELY after `git commit` returns 0 — the commit is durable on local disk regardless of remote push outcome; (B) push runs detached via `nohup ./push_all.sh ... > <log> 2>&1 & + disown` — the orchestrator's exit code reports COMMIT success, NOT push success; (C) `push_all.sh` acquires per-remote flock `.git/.push.<remote>.lock` so concurrent invocations targeting the same remote serialize but different-remote invocations run in parallel; (D) backgrounded push failures land in `qa-results/push_failures/<ts>_<remote>.log` — the next autonomous-loop tick checks per §11.4.87(A) “no external dependency in-flight” gate; (E) synchronous-push escape: explicit `--sync-push` CLI flag preserves legacy behaviour for §11.4.41 force-push merge-first audit paths. Gates CM-COVENANT-114-88-PROPAGATION + CM-BACKGROUND-PUSH-WIRED + paired §1.1 mutations. Synchronous push (without `--sync-push`) = §11.4 PASS-bluff at the execution layer.

Cascade requirement: This anchor (verbatim or by §11.4.88 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-88-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker — no escape hatch beyond `--sync-push` for force-push events. **Canonical authority:** constitution submodule Constitution.md §11.4.88 for the full mandate.

§11.4.89 — Background Test Execution Mandate (cascaded from constitution submodule §11.4.89)

Verbatim user mandate (2026-05-27): “Any tests we are executing, especially long test cycles, MUST BE performed in background in parallel with main work stream! This MUST NOT block our capabilities to work on queued workable items. Main work stream can be blocked or sit iddle only if absolutely needed and if it depends hard on results of some background execution.”

Symmetric anchor to §11.4.88 (background push) at the test-execution layer. Mandate: (A) long-running tests (>30 s expected: `pre_build`, `meta_test`, `test_all_fixes`, `recent_work_validate`, HelixQA banks, 4-phase cycles, full-suite retests, audio supervisors, dual-display recorders) MUST run via `nohup ... > <log> 2>&1 & + disown` with the log under a known dir (`qa-results/<test_id>_<ts>.log`); (B) the main stream proceeds to the §11.4.42 priority queue immediately; (C) hard-dependency gating — poll an exit-status file or `pgrep -af <test>` before steps that need the exit code, surfacing as §11.4.66 interactive options if the test is still running; (D) failures land in `<log>` files, the next loop tick checks; (E) foreground execution permitted ONLY for <30 s tests OR explicit

operator authorisation; (F) per-script flock serialises same-script invocations, different-script invocations parallel. Gates CM-COVENANT-114-89-PROPAGATION + CM-BACKGROUND-TEST-EXECUTION-WIRED + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.89 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-89-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker — no escape hatch beyond explicit per-invocation operator authorisation. **Canonical authority:** constitution submodule Constitution.md §11.4.89 for the full mandate.

§11.4.90 — Obsolete Status + Per-Item Obsolescence Audit (cascaded from constitution submodule §11.4.90)

Verbatim user mandate (2026-05-27): “Bug No 6 ... seems obsolete after latest request for new behavior ... mark obsolete tickets with some light gray background ... text - the description to be strikethrough styled ... review all existing open or resolved workable items if they are obsolete - not valid any more ... There MUST NOT be any mistake! No bluff is allowed of any kind!”

The §11.4.15 Status closed-set is extended with a terminal Obsolete (→ Fixed.md) value (orthogonal to Type per §11.4.16). Obsolescence reasons (closed vocabulary): superseded-by-design-change | superseded-by-later-mandate | feature-removed | duplicate-of | unsupported-topology. Every Obsolete heading MUST carry an ****Obsolete-Details:**** line (Since + Reason + Superseding-item + Triple-check evidence) within 8 non-blank lines. The §11.4.23 colorizer adds a cell-status-obsolete class — light-gray #E0E0E0 background + strikethrough description. Audit cadence: every release-gate sweep per §11.4.40 + §11.4.42; triple-check is non-negotiable per the operator mandate. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.21 / §11.4.23 / §11.4.33 / §11.4.34 / §11.4.40 / §11.4.42 / §11.4.66 / §11.4.71. Gates CM-COVENANT-114-90-PROPAGATION + CM-ITEM-OBSOLETE-DETAILS + CM-OBSOLETE-COLORIZER-WIRED + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.90 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-90-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.90 for the full mandate.

§11.4.91 — Summary-Doc Clarity Mandate (cascaded from constitution submodule §11.4.91)

Verbatim user mandate (2026-05-27): *“Summary docs - Issues_Summary some not clear one line descriptions - like ‘Composes with’ ... For each workable item we MUST HAVE clearly understandable meaning ... every team member can clearly understand what that particular workable item is exactly about! There cannot be misunderstanding or unclarity of any kind and no bluff allowed!”*

Every summary entry (Issues_Summary, Fixed_Summary, README doc-link, Status_Summary pages 1+2, all one-liners) MUST contain a self-contained meaningful description ≥ 6 words OR ≥ 40 chars naming SUBJECT + PROBLEM/GOAL. Forbidden one-liner anti-patterns: section labels (Composes with, Closure criteria, Fix direction, etc.); bare metadata fragments (Critical, Bug, In progress, etc.); section-marker echoes; a \$-letter alone. Generators (generate_issues_summary.sh / generate_fixed_summary.sh / update_readme_doc_links.sh / generate_status_summary.sh) MUST extract from the H1/H2 heading line per the §11.4.54 ATM-NNN convention, NEVER from arbitrary downstream text, and MUST refuse anti-pattern rows — emitting a (MISSING DESCRIPTION – fix source heading) placeholder with visual highlight. Gate CM-SUMMARY-CLARITY-DESCRIPTIONS scans every summary; an anti-pattern match = FAIL. Audit cadence: every §11.4.40 + §11.4.42 sweep.

Cascade requirement: This anchor (verbatim or by §11.4.91 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-91-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.91 for the full mandate.

§11.4.92 — Multi-Pass Change-Evaluation Discipline (cascaded from constitution submodule §11.4.92)

Verbatim user mandate (2026-05-27): *“Every change to the project or codebase we do MUST BE evaluated in several passes and in in-depth analysis for potential new issues or problems it can introduce! ... no bluff of any kind! After we do change or set of changes this mandatory steps MUST BE taken!”*

Every non-trivial change MUST pass a 5-pass evaluation BEFORE it is commit-ready: **(Pass 1)** main-task verification — change achieves the stated goal, captured-evidence per §11.4.5/§11.4.69; **(Pass 2)** regression-blast-radius analysis — enumerate every direct dependency, demonstrate no contract break; **(Pass 3)** cross-feature interaction analysis — audit parallel features sharing state/timing/hardware/shell environment; **(Pass 4)** deep-research validation per §11.4.8 — external precedent OR “NO external solution found — original work” + CodeGraph queries per §11.4.78/§11.4.79; **(Pass 5)** anti-bluff confirmation per §11.4 / §11.4.1 / §11.4.6 / §11.4.27 / §11.4.50 / §11.4.52 / §11.4.69 / §11.4.83 — no new bluff surface introduced. Each pass is documented (commit footers OR docs/ entries OR qa-results/ evidence). Only after all 5 passes complete may commit/push/test/ release proceed. Trivial exemption: typo / revision-bump / MD-export-regen IF zero source touched AND the commit message cites the exemption explicitly. Gates CM-COVENANT-114-92-PROPAGATION + CM-MULTI-PASS-EVALUATION-EVIDENCE + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.92 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-92-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.92 for the full mandate.

§11.4.93 — SQLite-Backed Single-Source-of-Truth for Workable Items (cascaded from constitution submodule §11.4.93)

Verbatim user mandate (2026-05-27): *“There MUST be single source of truth for all of our workable items - SQLite database ... proper scripts (we recommend Go programs) ... reduce a chance for sync to be broken ... generate always all docs from DB or to re-generate Db from all docs we have in opposite direction”*

The text-based Issues/Fixed/Summary/CONTINUATION constellation is converted to a SQLite-DB-backed single source of truth. Schema mandatory tables: items (atm_id PK + Type + Status incl. Obsolete + Severity + title + description ≥40 chars + created/modified + composes_with JSON + current_location); item_history (append-only audit per §11.4.34 By/Reason/ Evidence); obsolete_details (§11.4.90); operator_block_details (§11.4.21); firebase_metadata (§11.4.47); meta (schema version + last sync + integrity hash). A Go binary at cmd/workable-items/ provides sync md-to-db / db-to-md / diff / validate / add / close; bidirectional regen is byte-identical round-trip (closed-set

whitespace/section-order tolerance). `commit_all.sh` refuses on non-empty diff; `sync_issues_docs.sh` invokes the Go binary; pre-build runs `workable-items validate`. Anti-bluff: unit + integration + stress (1000-row insert + 10 concurrent writers) + chaos (mid-write SIGKILL + corrupt-DB recovery + disk-full) + paired §1.1 mutation + HelixQA Challenge CME-WORKABLE-ITEMS-001. The Go binary lives in the constitution submodule (`constitution/scripts/workable-items/`) per §11.4.74. Gates CM-COVENANT-114-93-PROPAGATION + CM-WORKABLE-ITEMS-DB-PRESENT + CM-WORKABLE-ITEMS-MD-DB-IN-SYNC + paired §1.1 mutations. (NOTE: the DB tracking rule is AMENDED by §11.4.95 — DB is TRACKED, not gitignored.)

Cascade requirement: This anchor (verbatim or by §11.4.93 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-93-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker — text-based-only trackers are a §11.4 PASS-bluff at the data-architecture layer. **Canonical authority:** constitution submodule Constitution.md §11.4.93 for the full mandate.

§11.4.94 — Zero-Idle Priority-First Parallel-By-Default Operating Mode (cascaded from constitution submodule §11.4.94)

Verbatim user mandate (2026-05-27): “We *MUST NEVER sit iddle / wait or sleep if there is possibility for us to work on something ... Always check if there is a possibility to work on something while we are not working actively on something! Pick always by priority - most critical workable items and other tasks MUST BE done first! ... Stay still / iddle if nothing is left to be done at all or waiting for something that is blocking us / you!!!*”

§11.4.94 binds §11.4.20 + §11.4.42 + §11.4.58 + §11.4.70 + §11.4.72 + §11.4.82 + §11.4.87 + §11.4.88 + §11.4.89 into a single always-on enforcement: (A) idle ONLY when every queued item is genuinely blocked on an external dependency (hardware / network upstream / build/test completion the conductor cannot accelerate) OR operator STOP OR §12 host-safety — “don’t see what to do” is NEVER valid; (B) before ANY wake/sleep the conductor MUST survey parallel-work feasibility per §11.4.42 + §11.4.72 + §11.4.87, identify non-contending items, and dispatch in parallel per §11.4.20/§11.4.70 (subagent) + §11.4.58 (PWU disjoint scope) + §11.4.89 (background long tests); (C) priority order MANDATORY — pick highest-severity + §11.4.72 audio-first the conductor can autonomously progress; (D) subagent-driven default for non-trivial; (E) background default for >30 s wall-clock

work via nohup+disown; (F) stability-preserving (composes with §11.4.92 multi-pass + §11.4.84 quiescence + §12.6–§12.9 host safety); (G) progress updates surfaced at milestone boundaries. Gates CM-COVENANT-114-94-PROPAGATION + CM-PARALLEL-WORK-AUDIT + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.94 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-94-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.94 for the full mandate.

§11.4.96 — Safe-Parallel-Work-With-Long-Build Catalogue + Mandate (cascaded from constitution submodule §11.4.96)

Verbatim user mandate (2026-05-27): *“Are there except AOSP build process any other active jobs being done at the moment? Can we work on something in parallel while build is in progress so we slowly cleanup our slate? ... do as much as possible work in background in parallel with main work stream and oreferably using subagents-driven approach!”*

An operational catalogue for the canonical long-running workload (multi-hour containerised build per §12.9). **SAFE during build:** (A) MD/docs work; (B) generator/helper script work under scripts/; (C) pre-build + meta-test gate authoring + paired §1.1 mutations; (D) on-device test scripts; (E) constitution submodule edits + push; (F) any submodule commit + push per §11.4.88; (G) read-only live-ADB probes (dumpsys/getprop/cat /proc/.../screencap/logcat); (H) subagent dispatch per §11.4.20/§11.4.70 + §11.4.84 quiescence; (I) web research + external API queries with §11.4.10 credentials; (J) workable-items DB ops per §11.4.93+§11.4.95; (K) backgrounded pre-build + meta-test execution per §11.4.89.

UNSAFE during build: (α) git checkout/reset --hard/clean -df on the source tree (use git worktree); (β) mass file deletes/renames under built source trees; (γ) submodule pointer updates affecting built artefacts; (δ) out/ mutations; (ε) make clean/m clobber/rm -rf out/; (ζ) container destruction; (η) disk-filling breaching §12.9 free-space minimum; (θ) §12 host-session-safety breaches.

Conductor responsibility: before EVERY pause point during a long build, consult the catalogue, identify (A)-(K) queue items per §11.4.42+§11.4.72, and dispatch ≥1 per §11.4.20/§11.4.70 subagent default + §11.4.89 background. “Build running, nothing

else to do” is NEVER true per §11.4.94+§11.4.96. Gates CM-COVENANT-114-96-PROPAGATION + CM-PARALLEL-WORK-DURING-BUILD-AUDIT + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.96 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-96-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.96 for the full mandate.

§11.4.97 — Maximum-Use-of-Idle-Time + Progress-Update Cadence (cascaded from constitution submodule §11.4.97)

Verbatim user mandate (2026-05-27): *“keep it working, we should do as much as possible, if not it all but as much as we can as long as there is iddle time! it MUST be used! ... keep us updated about all progress and all phisycal proofs and gathered data as you progress through all open workable items!”*

Operating-mode capstone strengthening §11.4.87 + §11.4.94 + §11.4.96: (A) every minute of conductor idle time during which work could autonomously progress AND is not genuinely blocked = a §11.4.97 violation; “as much as possible, if not it all but as much as we can” is operative — dispatch CONTINUOUSLY through the entire idle window, not just at scheduled wakes; (B) progress-update cadence — emit an operator-facing 1-line update at every commit landed / subagent return / constitutional anchor / captured evidence / milestone closure, no operator prompt required; (C) continuous physical-proof gathering per §11.4.5 + §11.4.6 + §11.4.69 — every autonomous closure cites captured-evidence (evidence path goes into the §11.4.93 item_history.evidence_path when the DB lands); (D) composes with §11.4.5/6/13/20/27/42/50/52/69/70/72/83/85/87/88/89/94/96; (E) the idle-only-when-blocked closed-set is unchanged from §11.4.94(A). Gates CM-COVENANT-114-97-PROPAGATION + CM-IDLE-TIME-AUDIT + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.97 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-97-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.97 for the full mandate.

§11.4.95 — Workable-Items SQLite DB Is TRACKED in Git, NEVER Gitignored (cascaded from constitution submodule §11.4.95)

Verbatim user mandate (2026-05-27): *“We shall not Git ignore our workable items SQLite DB since it is our single source of truth ... workable items SQLite DB regularly committed and pushed to all upstreams!”*

§11.4.93’s earlier “gitignored per §11.4.30” clause is AMENDED — the DB at docs/workable_items.db is TRACKED in git, NEVER gitignored. It IS authoritative source data, NOT a build artefact. Every workable-items sync md-to-db that mutates state MUST stage + commit + push the DB alongside the MD regen per §11.4.19 atomic-move + §2.1 multi-upstream push. A WAL-checkpoint (PRAGMA wal_checkpoint(TRUNCATE)) is required before commit-stage so the transient .db-wal + .db-shm sidecars (gitignored per §11.4.30) are safely discardable. The §11.4.77 regeneration mechanism does NOT apply — the DB IS the source. Destructive DB ops require §9.2 hardlinked-backup + operator authorization; §11.4.41 force-push merge-first applies if DB history ever needs rewrite. Gates CM-COVENANT-114-95-PROPAGATION + CM-WORKABLE-ITEMS-DB-TRACKED + paired §1.1 mutation.

Cascade requirement: This anchor (verbatim or by §11.4.95 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-95-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.95 for the full mandate.

§11.4.98 — Full-Automation Anti-Bluff Mandate (cascaded from constitution submodule §11.4.98)

Verbatim user mandate (2026-05-28): *“Make sure we have full automation testing of all scenarios with real bot, main group and users without any manual intervention or contribution of real user! Everything MUST BE fully automatic and autonomous! These tests MUST BE able to rerun endless times when needed! ... Make sure there is no false positives in testing! Every test and its results MUST obtain real proofs of everything working! No bluff is allowed!”*

Closes the manual-intervention gap (§11.4 / §11.4.2 / §11.4.5 / §11.4.50 / §11.4.85 / §11.4.87 / §11.4.89 / §11.4.94 did not explicitly forbid it). A live/integration/e2e/Challenge test that requires a human action during execution (typing a message, clicking UI, hand-triggering a webhook, attaching a file — anything beyond startup) is by definition a §11.4 PASS-bluff at the automation layer. (A) Every governed test — unit/integration/e2e/Challenge/stress/chaos/live — MUST be fully self-driving end-to-end, reporting PASS/FAIL/SKIP-with-reason without any further human action after startup. (B) Single permissible exception: one-time credential bootstrap performed OUTSIDE test execution (.env from vault, shell exports, OAuth at first install, MTPROTO session activation) — configuration, not test driving. (C) Live messenger/channel/agent tests: no “operator must type” prompts (drive programmatically via second account / webhook fixture / loopback); no hard-coded session UUIDs that collide with the active dev session (Herald 2026-05-28 claude --resume silent exit -1 lesson); no 60 s human-response windows (§11.4.50 determinism violation); re-runnability proof — PASS at -count=3 consecutive automated invocations with self-cleaning state; §11.4.98 obsolescence audit classifies every existing test COMPLIANT vs NON-COMPLIANT; no silent-skip-reported-as-PASS or stale-evidence-as-fresh. (D) With §11.4.85 + §11.4.89 + §11.4.87 + §11.4.94 forms a continuously-validated, non-flake, anti-bluff regime. (F) Manual-dependency tests not rewritten within 30 days graduate to §11.4.90 Obsolete citing §11.4.98.

Cascade requirement: This anchor (verbatim or by §11.4.98 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-98-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.98 for the full mandate.

§11.4.99 — Latest-Source Documentation Cross-Reference Mandate (cascaded from constitution submodule §11.4.99)

Verbatim user mandate (2026-05-28): *“Make sure we ALWAYS check against latest versions of services we use web / online docs before creating instructions! This situation is illustration of how we can misguide ourselves or get banned! ... These are mandatory rules / constraints and the result is consistency and safety of created instructions, guides and manuals!”*

Misguidance-by-stale-docs is the same severity class as a §11.4 PASS-bluff at the documentation layer (Herald 2026-05-28 case: a first-draft MTPProto guide recommended VoIP fallback numbers and omitted the recover@telegram.org pre-login email — both contradicted Telegram’s official docs + the gotd/td maintainer guide and could have caused a permanent account ban). Closes the gap §11.4.92 Pass 4 alludes to but does not mandate. (A) Before committing any operator-facing instruction/guide/manual/troubleshooting/setup doc, the author MUST: (1) fetch the LATEST official online documentation of the documented service/library via WebFetch / MCP / direct browsing — NEVER training data, memory, or prior committed docs; (2) cross-reference every instruction step against that source; (3) seek secondary authoritative sources (maintainer SUPPORT.md, official changelogs, vetted community FAQs) when the official source is sparse/silent; (4) cite source URLs + date in a ## Sources verified footer in the doc; (5) cite a Sources verified <date>: <urls> footer in the commit message. (B) Negative findings (gaps/silences/contradictions) MUST be documented explicitly. (C) Docs older than 6 months are STALE — re-verify before citing as operator authority, at every vN.0.0 release boundary, on service breaking-change announcements, or on operator error reports. (D) Risk-classified services (messaging, cloud APIs, payment systems, AI/LLM providers, code-hosting, package managers) carry a 90-day max staleness + explicit safety warnings. (E) Composes with but is INDEPENDENT of §11.4.92 Pass 4. (G) Commit missing either footer is BLOCKED at release-gate; stale-beyond-grace docs graduate to §11.4.90 Obsolete (Reason=stale-documentation).

Cascade requirement: This anchor (verbatim or by §11.4.99 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-99-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.99 for the full mandate.

§11.4.101 — Autonomous-Decision-Over-Blocking Mandate (cascaded from constitution submodule §11.4.101)

Verbatim user mandate (2026-05-28): “when working in endless working loop fully autonomously try to decide most properly about points which would block execution and wait for us. If we haven’t answered now work would be blocked whole night! If possible and if that will not cause any issues make proper and most reliable and safe decision so we achieve maximal efficiency and work gets fully done!”

In autonomous / endless-loop mode (per §11.4.87), the agent MUST minimize operator-blocking and make the safe, reliable, reversible decision itself so work is not stalled (e.g. overnight) waiting for input — §11.4.87 says keep working, §11.4.101 says HOW to clear the decision points. **Proceed-autonomously (closed-set, ALL must hold):** (a) the action is reversible OR has a captured pre-op backup per §9.2; (b) the safe choice is determinable from captured evidence per §11.4.6 (no guessing — LIKELY/probably/seems is NOT a determination); (c) a wrong choice's blast radius is bounded AND recoverable; (d) it composes with anti-bluff §11.4, host-safety §12, data-safety §9. **Block-only-when (BLOCK via the §11.4.66 interactive mechanism ONLY when ALL hold):** the action is irreversible AND high-blast-radius AND the safe choice cannot be determined from evidence — e.g. external-account state the agent cannot inspect, hardware it cannot access, destructive ops without backup, force-push (also §9.2 + §11.4.41), spending money or sending data to third parties. Operator-blocked per §11.4.21 is reached only after this rule fires AND the self-resolution-exhaustion audit completes. An unavoidable block parks one work unit — it does NOT pause the loop; the agent keeps progressing every non-blocked item in parallel per §11.4.87 + §11.4.94 (posing the question then going idle is a §11.4.94 + §11.4.97 violation). Classification: universal (§11.4.17).

Cascade requirement: This anchor (verbatim or by §11.4.101 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-101-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.101 for the full mandate.

§11.4.102 — Mandatory systematic-debugging activation + always-loaded skill-discovery + plugin-dependency availability (cascaded from constitution submodule §11.4.102)

Verbatim user mandate (2026-05-29): *“Make sure that we ALWAYS trigger / start the”/superpowers:systematic-debugging” skills when any issues happen! ... we MUST activate the skill(s) and make strongest efforts in full in depth analysis / debugging and determine root causes of all problem ... we MUST make sure that “/using-superpowers” skill is ALWAYS loaded, applied and used! All dependencies*

(plugins) that Claude Code or other market places are offering MUST BE installed if these are not already available for loading and use!”

Three cooperating invariants — the difference between guess-and-retry and investigate-to-root-cause-first. **(A) Mandatory systematic-debugging activation.** On ANY spotted issue / bug / test failure / gate failure / regression / misalignment / inconsistency / unexpected behaviour, the agent MUST activate superpowers:systematic-debugging (or the platform-equivalent structured-debugging discipline) **BEFORE proposing, writing, or applying any fix** — the **Iron Law: NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST**. Full four-phase arc: root-cause → pattern → hypothesis → implementation (the fix is designed only against the proven root cause). Guess-and-retry, symptom-patching, and re-running a failed test hoping it passes (“probably transient / flaky”) WITHOUT a completed investigation are §11.4.102 violations; calling a failure transient/flaky/intermittent/probably-timing without captured forensic evidence is simultaneously a §11.4.6 (no-guessing) and §11.4.7 (demotion-evidence) violation. **(B) Mandatory always-loaded using-superpowers.** superpowers:using-superpowers (or the platform-equivalent skill-discovery / capability-index discipline) MUST be loaded and applied at session start and consulted before any task — survey available skills before acting on ANY request; if ANY skill could apply (even at 1% relevance) it MUST be invoked rather than improvised from memory. **(C) Mandatory plugin / dependency availability.** Every skill plugin / marketplace package / capability dependency the project relies on MUST be installed + loadable BEFORE the dependent work proceeds; a missing plugin that blocks a mandated skill is a release-blocker until installed + confirmed loadable (confirm by observing the skill in the live capability list — install exit 0 ≠ skill loadable, per the §11.4.80 lesson). Composes with §11.4.4 / §11.4.6 / §11.4.7 / §11.4.8 / §11.4.43 / §11.4.70 / §11.4.82(A) / §11.4.92. Classification: universal (§11.4.17). No escape hatch — no --skip-systematic-debugging, --guess-and-retry-OK, --symptom-patch-permitted, --skip-skill-discovery, --plugin-optional, --missing-plugin-is-warning flag.

Cascade requirement: This anchor (verbatim or by §11.4.102 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-102-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.102 for the full mandate.